데이터구조 필기본 — 코넬

AVL · B-Tree · Hashing · Graph · Shortest Path · MST · String — 코넬식 회독노트

Ch 10 AVL Tree — 균형 이진 탐색 트리

자가-균형 BST · Balance Factor · 4가지 회전으로 O(log n) 보장

균형 트리 & AVL 정의 p.1-6

Q. BST 최악이 왜 O(n)? 균형이 왜 필요?

Q. balance factor bf(x)의 정의는?

Q. AVL 성질(bf 허용 범위)은?

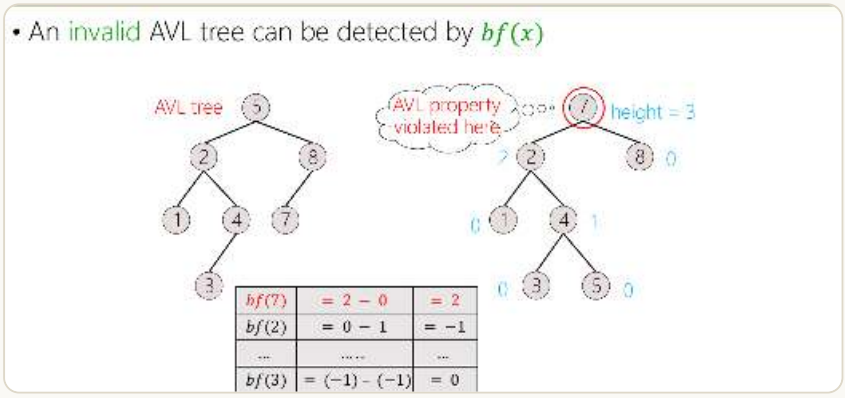
Q. pivot 노드란 무엇?

왜 균형이 필요한가 — BST 높이가 탐색·삽입·삭제 복잡도를 단독 결정. 정렬된 입력 → 링크드리스트로 퇴화 → O(n) 최악.

- 임의 BST: 평균 O(log n), 최악 O(n) (정렬 입력)
- 균형 BST: 최악과 평균 모두 O(log n)

AVL Tree — 자가-균형 BST. *모든 노드*의 두 서브트리 높이 차 ≤ 1. height-balanced BST.

- Balance Factor — `bf(x) = height(left(x)) − height(right(x))`
- 높이 관례: 잎 `height = 0`, 빈 트리 `height = −1`
- AVL 성질 — 모든 노드 x에 −1 ≤ bf(x) ≤ 1 성립
- |bf(x)| = 2인 노드가 *불균형 노드* = pivot 후보



AVL 유효·위반 예시 — bf(7)=2 위반, bf(2)=−1 유효 (p.6)

AVL 삽입 알고리즘 p.8-15

Q. 삽입 후 무엇을 잎→루트로 추적?

Q. pivot은 어떻게 식별?

Q. early termination의 근거는?

Pivot Node — 새 잎 → 루트 경로에서 bf = ±2가 되는 가장 깊은 노드. 삽입당 pivot은 최대 1개.

pivot = 가장 깊은 불균형 조상

- 경로 바깥 노드는 bf 불변 → pivot에서의 회전 1회로 전체 복구 가능
- Early termination — pivot 회전 후 서브트리 height가 삽입 전 값으로 복귀 → 조상 bf 불변 → 즉시 종료

AVL Insertion

```
// 1단계: BST 삽입
Insert key as leaf (standard BST rule)
// 2단계: 잎→루트 경로 추적
for each node x from leaf to root do
  if -1 ≤ bf(x) ≤ 1 then
    proceed to parent(x)
  else  ⟵ bf(x) = ±2, x is the pivot
    do appropriate rotation at x
    reconnect rotated subtree to parent(x)
    update heights of involved nodes
    return  ⟵ early termination
```

회전 후 서브트리 height = 삽입 이전 값 → 조상 bf 전혀 변하지 않음 → O(log n)

불균형 유형 4가지 p.13-14

Q. LL·RR·LR·RL 분류 기준은?

Q. 단일 vs 이중 회전 경우 구분?

분류 기준 — pivot A에서 새 노드까지의 *경로 방향*으로 결정. 4가지만 존재.

- LL — A 왼쪽 자식의 왼쪽 서브트리에 삽입 → 단일 오른쪽 회전
- RR — LL의 거울 → 단일 왼쪽 회전
- LR — A 왼쪽 자식의 오른쪽 서브트리에 삽입 → 이중 회전 RR+LL
- RL — LR의 거울 → 이중 회전 LL+RR

경우	조건(삽입 위치)	회전 유형	회전 수
LL	A.left.left 서브트리	Right rotation at A	1
RR	A.right.right 서브트리	Left rotation at A	1
LR	A.left.right 서브트리	Left at B, Right at A	2
RL	A.right.left 서브트리	Right at B, Left at A	2

A=pivot, B=A의 더 깊은 자식. 합성 항등식: LR=RR+LL, RL=LL+RR

단일 회전 — LL & RR p.16-21

Q. LL은 어느 방향 회전?

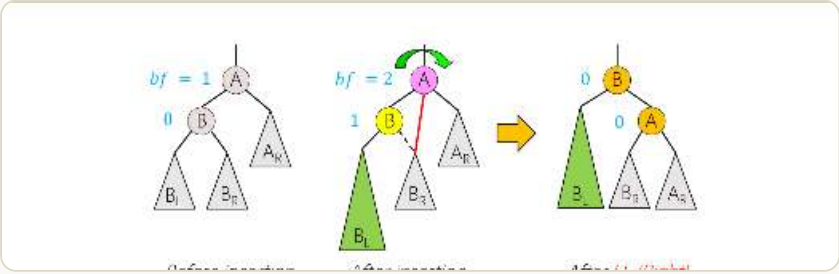
Q. RR은? 단일 회전이 언제 충분?

Q. 회전 후 subtree height는?

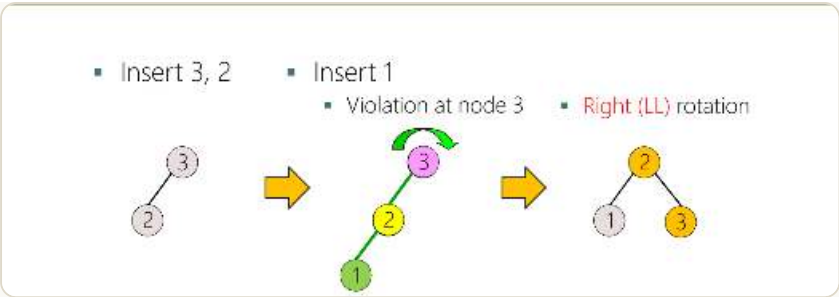
LL — 오른쪽 단일 회전 — pivot `A` (bf=2), 왼쪽 자식 `B` (bf=1). `B`를 새 서브루트로 올리고 `A`를 `B`의 오른쪽 자식으로 내림.

BR이 A.left로 이동

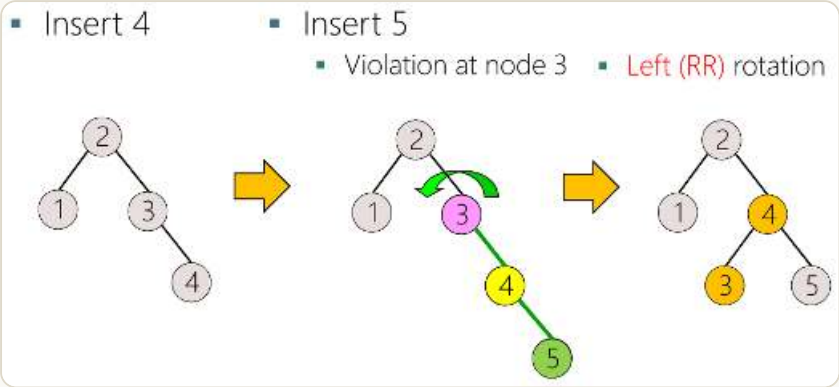
- `BR` (B의 오른쪽 서브트리)가 `A`의 왼쪽 자식으로 이동 — *유일한* 추가 포인터 변경
- 결과: `bf(B)=0`, `bf(A)=0` — 서브트리 height 삽입 전 값으로 복귀
- RR: LL의 거울. `A`의 오른쪽 자식 `B`를 올리는 왼쪽 회전



LL 구조 — Before/After insertion → Right rotation (p.17)



LL 예시 — 3,2,1 삽입 → pivot=3, Right(LL) rotation (p.18)



RR 예시 — Insert 4,5 → violation at node 3 → Left(RR) rotation (p.19)

⚠ **함정** LL은 왼쪽-왼쪽 경로 → 오른쪽 회전. RR은 오른쪽-오른쪽 → 왼쪽 회전. 경우 이름과 회전 방향이 반대라 헷갈림 주의.

이중 회전 — LR & RL p.23-28

Q. 단일 회전이 실패하는 이유는?

Q. LR = ? + ? (합성)

Q. RL = ? + ? (대칭)

왜 단일 회전이 실패하는가 — LR/RL은 새 노드가 *교차 방향* 서브트리에 도착. pivot A에서 단순 회전 시 `bf` 가 반대 방향으로 ±2가 되어 여전히 불균형.

- LR: A(bf=2), B=A.left(bf=−1) — 단일 오른쪽 회전 → B(bf=−2) 새 위반
- 해결: 1차 회전으로 LR 형태를 LL로 변환, 2차 회전으로 마무리

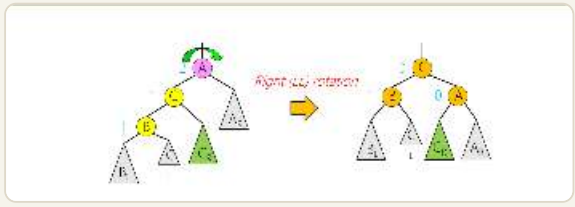
LR 이중 회전 = RR + LL — pivot `A` (bf=2) → 왼쪽 자식 `B` (bf=−1) → `B` 의 오른쪽 자식 `C` .

LR=RR+LL
RL=LL+RR

- 1단계 — B에서 왼쪽(RR) 회전: C가 올라와 A의 왼쪽 자식, B는 C의 왼쪽 자식으로 하강. A(bf=2) 아직 미해결.
- 2단계 — A에서 오른쪽(LL) 회전: C가 서브루트 자리 차지. B는 C.left, A는 C.right로 하강.
- 최종: bf(C)=0, bf(B)=±1, bf(A)=0/−1 — 서브트리 height 복귀
- RL = LL + RR: LR의 거울. B=A.right(bf=+1), C=B.left. 1차 LL at B, 2차 RR at A.

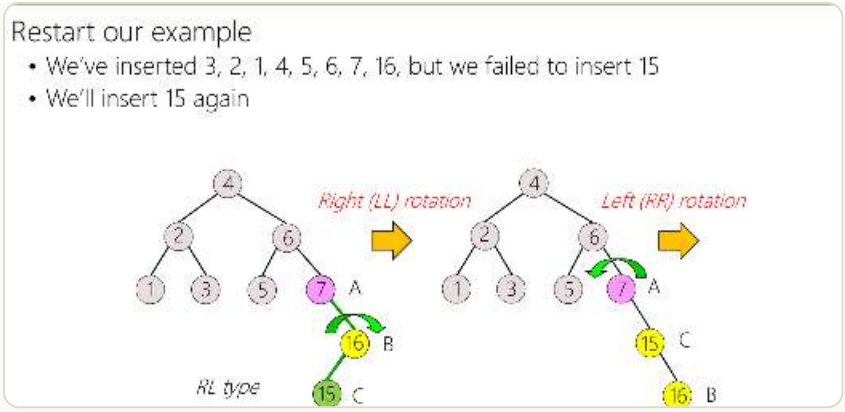


① 1차: Left(RR) at B → C 상승, A(bf=2) 미해결



② 2차: Right(LL) at A → C 서브루트, 균형 복원

LR 이중 회전 전체 과정 (p.25-26)



RL 예시 — Insert 15: A=7, B=16, C=15. RL = LL at A + RR at C (p.27)

LR type-1 vs type-2 — `C` 의 어느 서브트리에 새 노드가 도착했는지에 따라 최종 bf 값이 다름. 그러나 회전 시퀀스 자체는 동일 — RR+LL (LR), LL+RR (RL).

- type-1: C(bf=−1), 새 노드가 C.left에 → 최종 bf(B)=1, bf(A)=0
- type-2: C(bf=+1), 새 노드가 C.right에 → 최종 bf(B)=0, bf(A)=−1

AVL 삭제 알고리즘 p.36-46

Q. 삭제가 삽입과 다른 점(연쇄)?
Q. 동률(tie) 시 어느 회전 선택?
Q. early termination 있나?

BST 삭제 3가지 경우 복습 — AVL 삭제는 BST 삭제 + 재균형 루프.

- leaf 노드: 연결 해제
- degree-1: 단일 자식으로 교체
- degree-2: 키를 중위 successor(오른쪽 서브트리 최소값)로 교체 → 항상 leaf/degree-1 삭제로 환원

삽입 vs 삭제 재균형의 차이 — 삭제 후 회전이 서브트리 height를 줄일 수 있어 조상에서 연쇄 위반 발생 가능.

삭제=멈추지 않는다

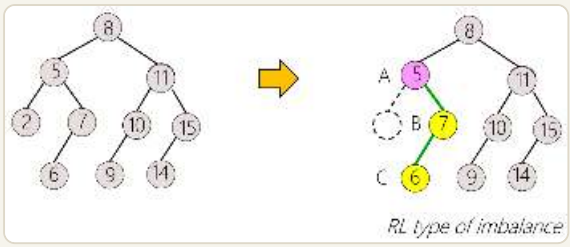
- 삽입: pivot 1회 회전 후 early termination — 최대 1회 회전
- 삭제: early termination 없음 — 루트까지 계속 진행, 최악 O(log n)회 회전

AVL Deletion

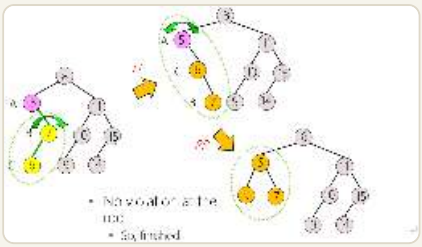
```
// 1단계: BST 삭제
Delete node by standard BST rules
// 2단계: 루트까지 경로 추적 (no early stop)
for each node x from deletion point to root do
  if −1 ≤ bf(x) ≤ 1 then
    proceed to parent(x)
  else  ⟵ bf(x) = ±2
    determine rotation type:
      B := child of x with greater height
      C := child of B with greater height
      path A→B→C spells the case (LL/RR/LR/RL)
    do appropriate rotation at x
    continue to parent(x)  ⟵ NO early termination
```

동률 규칙: B의 두 자식 height가 같으면 → single rotation 선택 (double이면 새 위반 생성 가능)

삭제 worked example — Delete 2 (leaf) — 트리 8(5(2,7(6,9)),11(10,15(·,14)))에서 leaf 2 삭제 → 노드 `5` 가 bf=−2 위반. pivot `A=5`, 더 깊은 자식 `B=7`, 그 자식 `C=6` → RL 유형 → RL=LL+RR 한 번으로 복원.



① Delete 2 → 노드5 bf=−2 (A=5,B=7,C=6) · RL 불균형

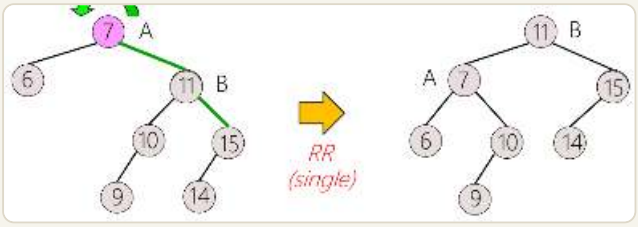


② RL(=LL+RR) 회전 → 6이 국소루트, root 위반 없음·종료

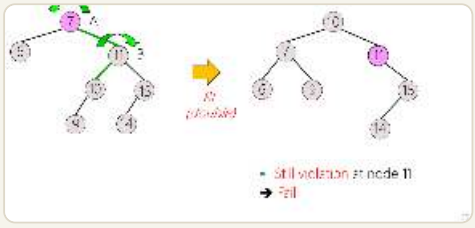
AVL 삭제 전체 과정 — BST 삭제 후 deletion point→root 추적, pivot에서 회전 (p.40-41)

동률(tie) 규칙 — B의 두 자식 height가 같을 때 반드시 single rotation 선택.

- single rotation: 서브트리 height가 1 줄어 위반 해소
- double rotation(오선택): 서브트리 하나가 더 깊어져 부모에 새 위반 → 연쇄 폭발



① RR 단일회전 선택 → 균형 복원 ✓



② RL 이중회전 선택 → 노드11 여전히 위반 ✗ Fail

Delete 5: B의 두 자식 height 동률 → single(RR)은 성공, double(RL)은 실패 (p.44-45)

복잡도 & 성능 분석 p.35-35, p.47-48

Q. AVL 연산 복잡도와 회전 횟수?

Q. AVL vs 일반 BST 언제 유리?

연산	BST (평균)	BST (최악)	AVL (평균/최악)	회전 수
Search	O(log n)	O(n)	O(log n)	0
Insertion	O(log n)	O(n)	O(log n)	최대 1회
Deletion	O(log n)	O(n)	O(log n)	최대 O(log n)회

AVL은 최악-평균 모두 O(log n) 보장. 삽입 단일회전·삭제 연쇄가 핵심 차이.

AVL vs 일반 BST — 실무 트레이드오프 — AVL이 최악 보장, BST가 평균-케이스 단순성.

- BST 유리: 작은 n (상수 인자 지배), 읽기 위주 워크로드(갱신 드묾), 무작위 입력
- AVL 결정적 승리: 정렬된 입력 — BST O(n) 붕괴, AVL O(log n) 유지
- AVL 비용: 모든 삽입·삭제마다 bf 갱신 + 가능 회전 = 상수 오버헤드

✎ 핵심 삽입 early termination이 가능한 이유: pivot 회전이 서브트리 height를 삽입 전 값으로 복원 → 조상 bf 불변. 삭제는 height를 줄이기 때문에 조상이 재위반 → 루트까지 진행 필수.

★ 필수암기 bf(x)=h(left)−h(right), AVL 성질: −1≤bf≤1 · 위반=|bf|=2=pivot · LL/RR=단일회전(1회) · LR=RR+LL · RL=LL+RR(2회) · 삽입=early stop(1회전) · 삭제=루트까지(O(log n)회전) · 전체 O(log n)

요약 AVL = 최초의 자가균형 BST, 모든 노드 **|bf|≤1** · 삽입=pivot에서 회전 1회 + **early termination** · 삭제=루트까지 연쇄 회전 · LL/RR=단일회전, **LR=RR+LL** · **RL=LL+RR** 이중회전 · 모든 연산 O(log n) 보장.

Ch 11 B-Tree — 균형 다원 탐색 트리

디스크 I/O 최소화 · Shape+Search Property · 삽입 split · 삭제 borrow/merge

동기 — 왜 다원 탐색 트리인가 p.1-6

Q. 디스크 I/O 비용 모델(접근≪이동)?

Q. 다원 노드가 I/O를 줄이는 이유?

Q. 메모리 기반 vs 디스크 기반 트리?

노드=블록 I/O 단위

디스크 I/O 비용 모델 — 메모리 접근 = *나노초* · 디스크 접근 = **밀리초**. 방문 노드 수 = I/O 횟수.

- BST: $O(\log_2 n)$ 노드 방문 → $O(\log_2 n)$ I/O
- **다원(m-way) 노드**: 한 노드 = 디스크 블록 1개 → 블록 1회 읽기로 *노드 전체* 획득
- 차수 m=100이면 $\log_{50} 10^9 \approx 4.3$ — 검색당 4~5회 I/O로 끝남

균형 탐색 트리 두 계열 — 저장 매체에 따라 분류. AVL·Red-black·Splay = *메모리 기반*. B-tree 계열 = **디스크 기반**.

- 메모리 기반: AVL, Red-black, Splay, 2-3, 2-3-4
- 디스크 기반: B-tree, B+-tree, B*-tree, Prefix B-tree

B-Tree 구조 — Shape & Search Property p.3-7

Q. Shape Property 3조항은?

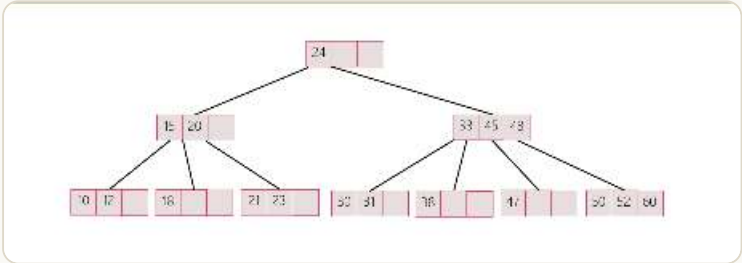
Q. Search(Tree) Property는?

Q. 높이 보장 O(log n)?

자식 수 [⌈m/2⌉, m]

B-Tree (차수 m) 정의 — Shape Property를 만족하는 *m-way* 탐색 트리.

- **Shape Property** 3조항: ① 루트는 자식 ≥ 2 · ② 루트 외 내부 노드는 자식 ≥ $\lceil m/2 \rceil$ · ③ **모든 잎은 동일 깊이**
- **Search Tree Property**: 자식 i의 모든 키 ∈ [부모 k_i, 부모 k_{i+1}) — 각 키가 범위 경계(분리자)로 작동
- 높이 보장: $O(\log_{\lceil m/2 \rceil} n)$ → 검색·삽입·삭제 모두 O(log n)



B-tree order 4 예시 — Shape+Search Property 동시 만족 (p.5)

2-3 트리 (차수 3) — B-tree의 가장 작은 비자명 인스턴스. 내부 노드가 *2 또는 3* 자식.

- 잎 수 범위: $2^h \le n \le 3^h$ → $\log_3 n \le h \le \log_2 n$ = O(log n)
- 자식 수 하한 2가 핵심 — 1이면 사슬 퇴화 가능, **2이면 편향 불가**
- BST는 정렬 입력 시 h=Θ(n) 퇴화 가능; 2-3은 불가 → 최악/평균 모두 O(log n)

Search p.14

Q. 노드당 비교 후 어느 자식으로?

Q. B+에서 내부 노드 키의 의미는?

2-3 Tree Search(x, node)

```
if x < k1:   return search(x, node.left)
if x = k1:   return node.record(k1)
if k1 < x < k2:   return search(x, node.middle)
if x = k2:   return node.record(k2)
if x > k2:   return search(x, node.right)
```

노드당 최대 2회 비교 후 유일 자식으로 하강. 경로 길이 = h → O(log n). B+ 스타일은 항상 잎까지 내려간다(분리자에서 발견돼도).

⚠ **함정** B+ 트리에서 내부 노드의 키는 *분리자*일 뿐 — 실제 레코드는 **잎에만** 존재. 내부에서 키 발견해도 잎까지 내려가야 한다.

Insertion — 분할(Split)과 승격(Promote) p.16-26

Q. 가득 찬 잎은 어떻게 처리?
Q. promote(승격)란?
Q. 트리가 자라는 유일한 경로는?

분할=삽입
승격=위로

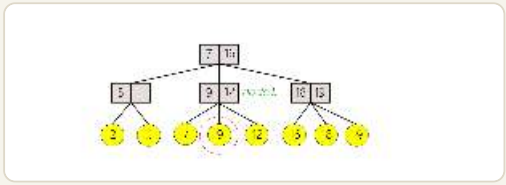
삽입 알고리즘 — 찾기 → 직접 삽입 or 분할-승격 연쇄.

- Step 1: Search Tree Property로 대상 부모 L 탐색
- Step 2: L이 가득 차지 않았으면 → 직접 삽입 (연쇄 없음)
- Step 3: L이 가득 찼으면 → L을 L과 L'으로 분할 · 키 균등 재배치 · L'의 최솟값 키를 부모로 승격
- 연쇄: 승격 키가 부모도 채우면 같은 분할-승격을 한 레벨 위로 반복
- 루트 분할: 연쇄가 루트까지 오면 새 루트 생성 → 높이 +1 (트리가 자라는 유일한 경로)

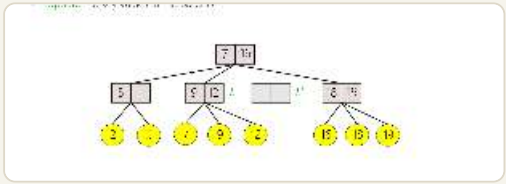
2-3 Tree Insert(key)

```
L ← 부모 노드 찾기(key를 자식으로 두게 될 노드)
if L is not full:  L에 key 직접 삽입
else:
  분할: L → (L, L')  [키를 균등 재배치]
  promote ← L'의 최솟값 키
  Update 분리자 in parent(L)
  if parent(L) is full:  recurse Insert(promote, parent(L))
  else:  parent(L)에 promote 흡수  ← 연쇄 종결
if 루트 분할:  new_root = {promote}; height ← height + 1
```

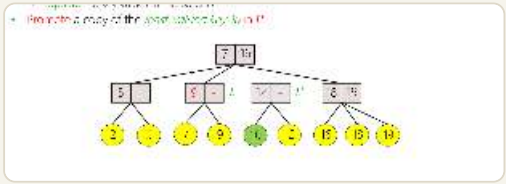
비용: O(h) = O(log n). 분할당 O(1). 새 루트는 항상 비어있어 연쇄 종결 보장.



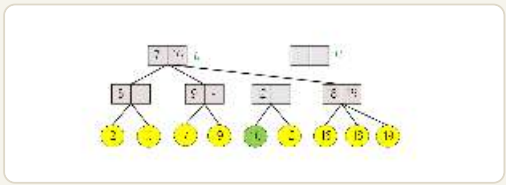
① key=10 삽입, L=[9,12] 가득참



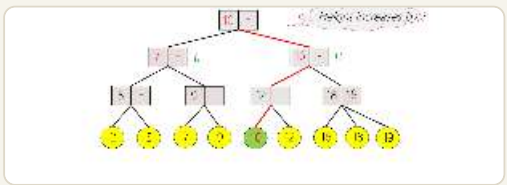
② L 분할 L·L', 키 재배치



③ 12 승격, 루트도 가득참



④ 루트 분할, L과 L' 생성



⑤ 새 루트 생성, h+1

삽입 key=10 — 이중 연쇄 split·promote 전체 과정 (p.19-26)

✎ 핵심 트리는 루트에서만 자란다. 루트 분할이 유일한 높이 증가 경로. AVL의 회전 없이 Shape Property가 편향을 구조적으로 차단.

Deletion — Borrow / Merge p.28-40

Q. 삭제 3가지 경우(A/B/C)는?
Q. borrow vs merge 차이?
Q. 루트가 줄어드는 조건은?

삭제 알고리즘 — 3경우 — 부모 N을 찾고 자식 수에 따라 분기.

A: 직접
B: borrow
C: merge

- Case A — N이 3자식: 단순 제거 → N은 2자식 유지 · Shape 보존
- Case B — N이 2자식 + 형제에 여유: 형제에서 빌리기(borrow) + 분리자 재작성
- Case C — N이 2자식 + 여유 형제 없음: N의 키를 형제로 이전 + N 병합(merge) + N 재귀 삭제 → 연쇄 가능

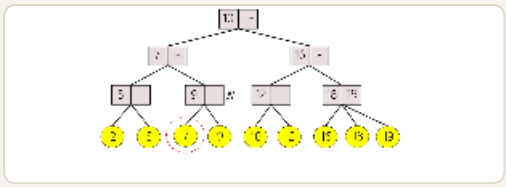
경우	조건	동작	연쇄	높이
Case A	부모 3자식	직접 제거	없음	불변
Case B	부모 2자식·형제 여유	borrow + 분리자 재작성	없음	불변
Case C	부모 2자식·여유 형제 없음	이전 + merge + 재귀 제거	있음	줄 수 있음

삭제 3경우 비교. Case C만 연쇄 → 루트 제거 시 height−1

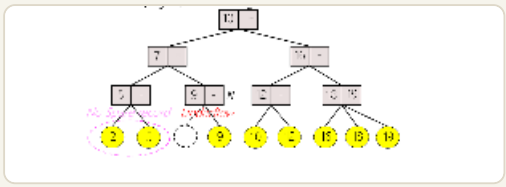
2-3 Tree Delete(key)

```
N ← key를 자식으로 둔 부모 노드 탐색
if N has 3 children:                ← Case A
  제거 후 분리자 재작성
else:                               ← N has 2 children → Underflow
  if 인접 형제에 여유 자식:          ← Case B
    borrow: 형제에서 키+자식 이전
    분리자 재작성
  else:                             ← Case C
    N의 키를 형제로 이전
    N을 트리에서 제거
    if parent(N) underflows:  recurse Delete(parent(N))
if 루트 자식=1:  루트 제거, height ← height − 1
```

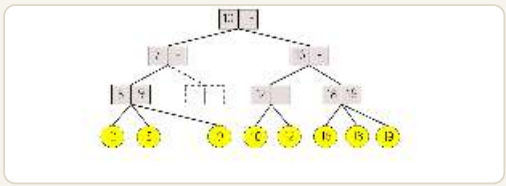
비용: O(h). borrow=O(1). merge 연쇄=레벨당 O(1). 루트에서 종결 보장.



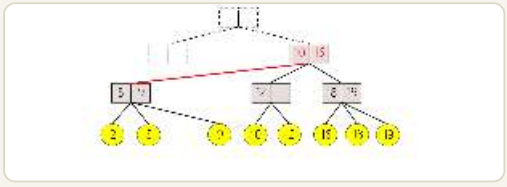
① key=7 삭제 대상 탐색



② N underflow·형제 여유 없음



③ key=9 형제로 이전·N 제거



④ 병합 완료·분리자 재작성

삭제 key=7 — Case C 병합 전체 과정 (p.36-40)

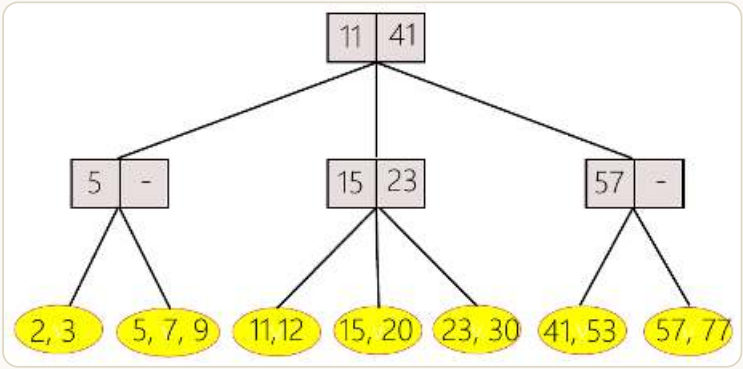
⚠ **함정** 삭제에서 Underflow = 자식이 최소(2) 미만. borrow 시도 → 실패하면 merge. merge 후 부모도 underflow면 연쇄 발생. 루트 자식=1이 되면 루트 제거 → 높이 −1(축소의 유일한 경로).

B+ Tree & 확장 p.7-9

Q. B vs B+ vs B* 핵심 차이?

Q. B+가 DB 표준인 이유는?

Q. 범위 스캔 복잡도는?



B+ tree order-3 예시 — 내부=분리자, 잎=레코드+연결 (p.10)

B vs B+ vs B* — 세 변형의 핵심 차이.

- B-tree: 데이터를 *모든* 노드에 저장 · 키 중복 없음 · 조기 반환 가능
- B+ tree: 데이터를 잎에만 저장 · 내부 노드 fan-out ↑ · 잎 링크 → *범위 스캔* O(1) 추가
- B* tree: B+ 변형 · 노드 채움률 ≥ 2/3 · 오버플로 시 형제 재분배로 분할 지연

	B-tree	B+ tree	B* tree
데이터 위치	모든 노드	잎만	잎만
키 중복	불허	허용(분리자↔잎)	허용
범위 스캔	O(n) DFS	O(k) 잎 링크	O(k) 잎 링크
채움 보장	≥ ⌈m/2⌉/m	≥ ⌈m/2⌉/m	≥ 2m/3
실무 채택	드묾	DB 표준	고밀도 특수 용도

B 계열 비교. 현대 DB는 B+ 채택 — fan-out·범위스캔 우위

✎ **핵심** B+ 표준이 된 이유: ① 내부 노드가 분리자만 → fan-out 극대 ② 잎 링크 → 범위 스캔 선형 ③ 모든 검색 동일 루트-잎 길이 → 비용 예측 ④ 변경이 잎에서만 발생

성능 분석 & 정렬 배열 비교 p.41-43

Q. 성장·축소 지점은?

Q. B-tree vs 정렬배열 트레이드오프?

성장/축소 지점 — 2-3 / B-tree는 항상 루트에서 자라고 루트에서 줄어든다.

- 삽입 연쇄 종점 = 루트 분할 → 새 루트(단일 키)로 종결 → h+1
- 삭제 연쇄 종점 = 루트 자식=1 → 루트 제거 → h−1
- AVL처럼 회전 불필요: Shape Property가 편향 사슬 자체를 *구조적으로 차단*

B-Tree vs 정렬 배열 — 트레이드오프 비교.

- 정렬 배열 *장점*: 공간 효율 ↑ · 이진 검색 메모리 접근 O(log n)
- 정렬 배열 *단점*: 삽입/삭제 O(n) (이동 비용) · 디스크에서 binary search = 무작위 I/O
- B-tree *장점*: 삽입/삭제 O(log n) · 노드=블록으로 디스크 접근 최소화 (m=100 → 10^9개에 4~5 I/O)
- 선택 기준: 정적·읽기전용·메모리 → 정렬 배열 / 동적·디스크·갱신 → B-tree

★ **필수암기** B-tree: Shape(루트≥2자식·비루트≥⌈m/2⌉자식·잎동일깊이)+Search Property · h=O(log n) · 삽입=분할+승격연쇄·루트분할로h+1 · 삭제=Case A(직접)/B(borrow)/C(merge연쇄·루트제거로h−1) · B+는 잎에만 데이터·범위스캔O(k)·DB표준 · 2-3트리: $2^h \le n \le 3^h$

요약 B-tree = 디스크 기반 m-way 탐색트리(Shape: 루트≥2자식·내부≥⌈m/2⌉자식·잎 동일깊이) + Search Property · 삽입=**split+promote** 연쇄→루트분할 시 h+1 · 삭제=A 직접/B borrow/C merge→연쇄 시 h−1 · B+=데이터 잎에만+잎 링크로 범위스캔 O(1)·**DB 표준**.

Ch 12 Dictionary & Hashing — 딕셔너리와 해싱

key-value 검색 · hash function · 충돌 해결 · 클러스터링 · rehashing

Dictionary ADT & 구현 계열 비교 p.1-9

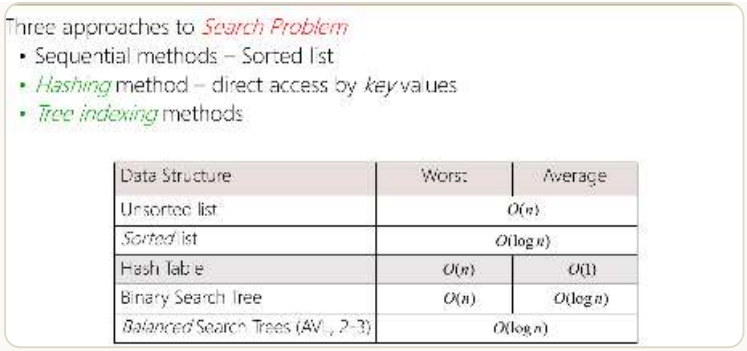
Q. Dictionary 핵심 3연산은?

Q. 구현 계열 3가지와 복잡도는?

Q. hash O(1)의 함정(가정)은?

Dictionary ADT — key-value pair `(k, i)` 의 집합. 핵심 연산 세 가지: insert·find·delete.

- key는 dictionary 내 유일 — 중복 key는 *교체*로 처리
- find(k): key 일치 item 반환, 없으면 miss
- 구현 계열 세 가지 — sequential / hashing / tree indexing



Dictionary 구현 계열 복잡도 비교 (p.3)

복잡도 비교 요약 — Hash table만이 Average O(1)을 달성. Worst는 O(n) (모든 key 충돌).

hash=평균 최속
tree=worst 보장

- Unsorted list: Worst/Avg = O(n) / O(n)
- Sorted list (이진탐색): Worst/Avg = O(log n) / O(log n)
- Hash table: Worst = O(n), Average = O(1)
- AVL / 2-3 tree: Worst/Avg = O(log n) / O(log n) — worst-case 보장

> ⚠ **함정** Hash O(1)은 *simple uniform hashing* 가정의 기대값. adversarial input이나 편향된 h(·)는 O(n) worst를 유발. *range query*는 hash가 순서 정보를 잃어 tree에 밀림.

Hashing 개요 — hash function `h : key → {0,..,M−1}` 로 slot 주소 직접 계산 — 포인터 추적·비교 없이 배열 index 직접 접근.

- 핵심 압축: 희소 key 도메인(예: 9자리 학번 = 10^9)을 크기 M table로 압축
- 이상적 h = *균일 분산* + *계산 용이* (두 마리 토끼)
- $h(k_1) = h(k_2)$, $k_1 \neq k_2$ → collision (비둘기집 원리로 불가피)

Hash Function 설계 p.10-25

Q. hash table 설계 4가지 이슈?

Q. division에서 짝수 M이 나쁜 이유?

Q. folding·mid-square란?

4가지 설계 이슈 — hash table 설계가 답해야 할 독립적이지 않은 네 축.

- ① hash function h 선택
- ② collision 처리 정책
- ③ table 크기 M 결정
- ④ overflow (table 꽉 찼을 때) 처리

네 가지 hash function 계열 — 각 계열의 key→slot 메커니즘.

- Division — `h(x) = x mod M` ; 구현 최단, M 선택에 민감
- Folding — key를 chunk로 나눠 합산 후 mod; 문자열 key에 자연스럽게 일반화
- Mid-square — key^2 계산 후 중간 r bit 추출 (M=2^r 시 최적)
- Digit analysis — 알려진 key 집합에서 편향 digit 제거 후 사용

Division Hashing

```
int hash(int x) {
  return (x % M);
}
```

한 줄 구현. 균일성은 전적으로 M 선택에 의존.

Division — M 선택 규칙 — M이 나쁘면 key 패턴이 residue에 각인되어 분포 편향.

M=짝수→X
M=소수→O

- 짝수 M 금지: h(x)=x mod M이 parity 보존 → 홀수 key는 홀수 slot만, 테이블 절반 낭비
- 홀수지만 작은 소인수 있는 M도 나쁨 — 예: M=651=3×7×31
- 큰 소수 M 권장: gcd(d, M)=1 보장 → 공통 인수 차단, full spread

소수 M이 필요한 이유 — key가 패턴을 가질 때(예: 10, 20, 30, ⋯) 소수 M만이 full spread를 보장.

- Key mod 4 → {2,0,2,0,2} (2종류만) vs Key mod 7 → {3,6,2,4,1} (5종류)
- 일반 규칙: key와 M이 공통 인수 d 공유 시 residue가 gcd 배수로 한정
- 소수 M: 제수가 1과 자기 자신뿐 → gcd 항상 1 → 함정 원천 차단

Folding — 문자열/정수 key — key를 동일 크기 chunk로 분리해 합산 후 mod. 모든 digit이 결과에 기여.

- 예: 123456789, M=10 → chunks 123,456,789 → 합 1368 → 1368 mod 10 = 8
- 단순 합은 *교환 대칭* — 'abc'='cba'. Fix: polynomial rolling hash $\Sigma c_i \cdot B^{t-i}$

Mid-square — key를 제곱 후 중간 r bit 추출. M=2ʳ일 때 정수 key에 탁월.

- 예: 4567^2 = 20,857,489 → 중간 2 digit → 57
- 중간 비트는 모든 digit 쌍의 cross-product 반영 → 단일 digit 편향이 분산됨

Collision Resolution — 충돌 해결 p.26-36

Q. open vs closed 용어 주의점?

Q. separate chaining의 특성은?

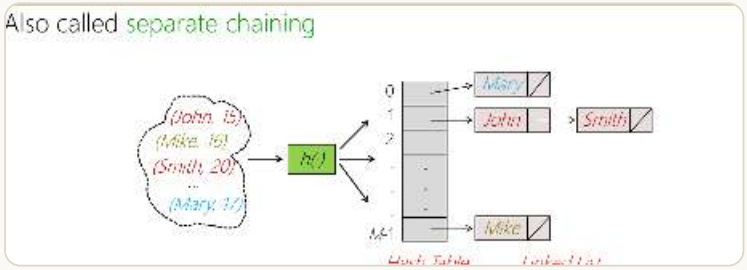
Q. closed의 3-state 모델은?

Static vs Dynamic Hashing — h·M 고정 여부로 분류.

- Static hashing: h, M 고정. collision → open(chaining) or closed(open addressing)
- Dynamic hashing: M 성장 가능 (extensible, linear hashing). DBMS에 적합
- 용어 주의: open = separate chaining, closed = open addressing (교과서마다 뒤집힘!)

Separate Chaining (Open Hashing) — 각 slot에 linked list를 붙여 충돌 key를 chain에 저장.

- load factor α = n/M = *기대 chain 길이* — α > 1도 허용 (chain 무한 성장)
- delete: list 노드 제거, *deleted marker 불필요*
- main memory + in-memory linked list 환경에 적합



Separate Chaining 구조 (p.27)

Separate Chaining — Insert/Find

```
Insert(k, item):
  i := h(k)
  chain[i].prepend((k, item))     ← chain 앞에 추가 (O(1))
Find(k):
  i := h(k)
  for each (key, val) in chain[i] do
    if key == k then return val
  return MISS
```

평균 chain 길이 = α. load factor 낮게 유지 시 O(1) 기대.

Closed Hashing (Open Addressing) — 3-state 모델 — 모든 record를 table 내부에 저장. 외부 chain 없음. 각 slot은 세 상태.

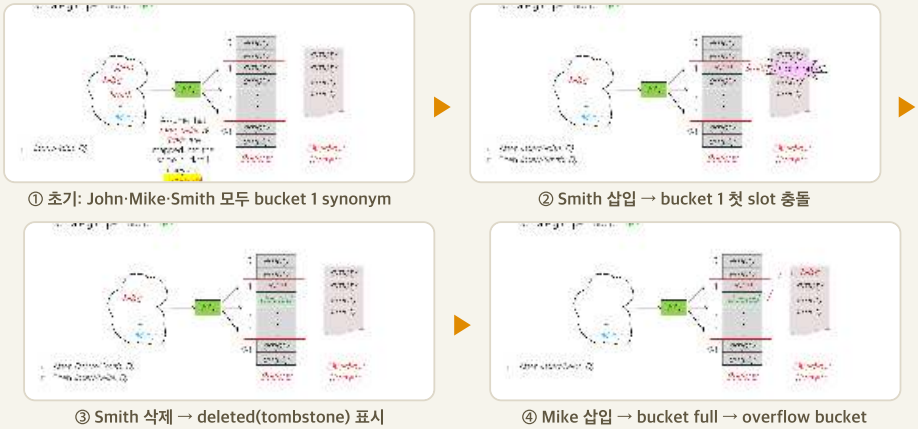
empty=포기
deleted=계속
occupied=비교

- empty: 미사용 — search *중단* 신호
- occupied: 현재 (k, item) 보유
- deleted (tombstone): 삭제됨 — search *계속* 신호. 없으면 거짓 miss!

⚠ 함정 Delete 후 slot을 empty로 바꾸면 그 slot을 넘어 배치된 key가 search miss 됨. 반드시 deleted(tombstone)으로 표시해야 probe가 계속된다.

Bucket Hashing — M slot을 고정 크기 bucket b로 분할 + 공유 overflow bucket.

- bucket 크기 b = disk block 크기에 맞추면 → 한 번의 disk I/O로 bucket 전체 로드
- 삽입: bucket 내 빈 slot → 없으면 **overflow bucket** (모든 bucket 공유)
- 삭제: slot을 deleted로 표시 → search invariant 유지



① 초기: John·Mike·Smith 모두 bucket 1 synonym ② Smith 삽입 → bucket 1 첫 slot 충돌 ③ Smith 삭제 → deleted(tombstone) 표시 ④ Mike 삽입 → bucket full → overflow bucket

Bucket Hashing 삽입·충돌·삭제·overflow 전체 과정 (p.29-35)

Open Addressing & Probe Function p.37-45

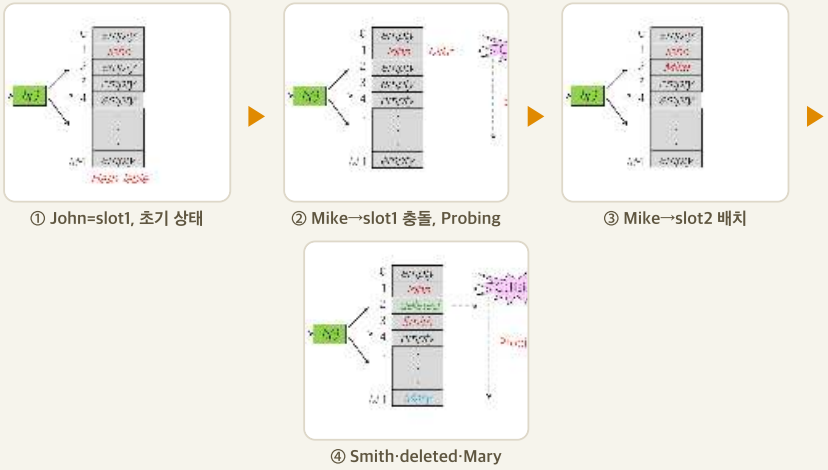
Q. probe function 일반식은?

Q. linear·quadratic·double의 p(i)?

Q. primary vs secondary clustering?

Probe Function 일반 공식 — 충돌 시 다음 slot을 결정하는 규칙. p(i)의 선택이 전략을 완전히 결정. p(i)=전략 파라미터

- $h_i(x) = (h(x) + p(i)) \bmod M$, p(0)=0 (첫 시도는 항상 home slot)
- **Linear probing**: p(i) = i
- **Quadratic probing**: p(i) = i^2
- **Double hashing**: p(i) = $i \cdot h_2(x)$



① John=slot1, 초기 상태 ② Mike→slot1 충돌, Probing ③ Mike→slot2 배치 ④ Smith·deleted·Mary

Open Addressing 삽입 전체 과정 — collision·probing·deleted marker (p.38-41)

Linear Probing — **p(i) = i** — 충돌 시 +1, +2, … 순서로 선형 스캔.

- Primary clustering: 연속 점유 run이 그 안에 home이 있는 모든 후속 key를 끌어들임
- Cluster가 길수록 다음 insert가 run을 extend할 가능성 상승 → 양의 피드백
- 장점: cache locality — 연속 메모리 스캔, production 기본값 많음
- stride c 변형 `p(i)=c·i` : gcd(c,M)=1 필요, 그래도 clustering 해소 불가

Quadratic Probing — $p(i) = i^2$ — probe gap이 1, 4, 9, 16, ⋯ 으로 성장 → primary cluster를 jump over.

- Primary clustering 제거. 그러나 secondary clustering 잔존
- 같은 home h(k) 가진 두 key → 동일 probe 수열 공유
- M=2의 거듭제곱이면 모든 slot 방문 실패 → 소수 *M* 또는 $\pm i^2$ 교대 사용

Secondary Clustering — probe 수열이 key가 아닌 *home position만의 함수*일 때 발생.

- $h(k_1)=h(k_2)$인 두 다른 key가 완전히 동일한 probe 수열을 따름
- Fix: probe 수열이 key x 자체에 의존하게 해야 함 → double hashing
- 계층: primary(최악) → secondary(중간) → 없음(double hashing)

Double Hashing — $p(i) = i \cdot h_2(x)$ — 두 번째 hash function `h₂(x)` 로 probe 단계를 key별로 modulate.

- $h_i(x) = (h(x) + i \cdot h_2(x)) \bmod M$
- 같은 h(x)라도 $h_2(x)$가 다르면 다른 probe 수열 → secondary clustering 제거
- 제약: $h_2(x) \neq 0$ (freeze 방지), $\gcd(h_2(x), M)=1$ (모든 slot 방문)
- 비용: 연산당 h 평가 *두 회* — secondary clustering 제거의 가격

전략	p(i)	Primary Cluster	Secondary Cluster	비고
Linear	i	있음	없음	cache 친화적
Quadratic	i^2	없음	있음	소수 M 권장
Double hash	$i \cdot h_2(x)$	없음	없음	$h_2 \neq 0$, gcd=1 조건

Open Addressing 전략 비교

Rehashing & Load Factor p.36·p.52-53

Q. load factor α 정의와 임계는?

Q. rehashing이 amortized O(1)인 이유?

Q. tombstone(deleted)의 역할은?

Load Factor $\alpha = n/M$ — table 점유 비율. α가 작을수록 평균 probe 수가 줄어든다.

- Open addressing: $\alpha < 1$ 유지 필수 ($\alpha \geq 1$이면 insert loop 미종료)
- Separate chaining: $\alpha > 1$도 허용 (chain이 무한 성장)
- 통상 임계: 0.5~0.75에서 rehashing 트리거

Rehashing — α가 임계를 넘으면 더 큰 새 table을 할당하고 *live record만* 재삽입.

O(n) 1회 → amortized O(1)

- tombstone은 버림 — 재삽입 대상 아님 (load factor 부풀림 해소)
- 1회 비용 O(n), 하지만 n번 insert에 1번꼴 → amortized O(1)
- primary clustering + tombstone 누적 두 문제를 동시에 해결

✎ **핵심** Rehashing이 amortized O(1)인 이유: rebuild 비용 O(n)을 그 이후 O(n)번의 insert에 분산하면 연산당 O(1). 임계 너무 작으면 rebuild 빈도↑, 너무 크면 probe 길이↑.

Pseudo-Random Probing & Shift-Register p.45-48

Q. pseudo-random probing 원리는?

Q. full-period 수열 조건은?

Pseudo-Random Probing — 고정 random permutation $\{d_1, d_2, \cdots, d_{M-1}\}$를 probe 수열로 사용.

- $h_i(x) = (h(x) + d_i) \bmod M$ — permutation 공유 필요
- Primary clustering 제거. 그러나 수열이 home position의 함수 → secondary 잔존
- Shift-register sequence: $M=2^k$, 상수 k, 시작값 d_1으로 결정적 permutation 생성

Shift-Register Recurrence

```
d₁ := 임의 시작값 ∈ [1, M−1]
for i := 2 to M−1 do
  d := d * 2              ⟵ 1-bit left shift
  if d >= M then d := d − M
  dᵢ := d XOR k           ⟵ 상수 k와 bitwise XOR
```

full-period 수열 = GF(2) 위 primitive polynomial 조건 만족하는 k. M=8, k=3, d_1=5 → 5,1,2,4,3,⋯

성능 요약 & Adversarial Input p.52-53

Q. chaining vs open addressing 비교?

Q. worst case와 방어책 (universal)?

	Separate Chaining	Open Addressing
저장 위치	table 외부 chain	table 내부
α 범위	> 1 허용	< 1 필수
delete	list 제거 (간단)	tombstone 필요
clustering	없음	linear→primary, quad/dh→secondary/없음
적합	메인메모리, 포인터 cheap	캐시 친화, 연속 메모리

Separate Chaining vs Open Addressing 비교

Adversarial Input & Worst Case — hash table의 평균 O(1)은 *simple uniform hashing* 가정의 기대값.

- Worst case: 모든 key가 같은 slot으로 hash → O(n) probe
- Hash-flooding 공격: 의도적으로 충돌 key를 보내 상수 시간을 선형 시간으로 전환
- 완화책: Universal hashing(랜덤 family, 공격자가 예측 어려움) 또는 balanced tree

★ 필수암기 Dictionary=insert·find·delete · h: key→[0,M-1] · 4이슈(h선택/충돌/M크기/overflow) · M은 큰 소수 · Chaining=외부list·α>1허용·delete간단 · OpenAddressing=내부·α<1·tombstone · Linear=primaryCluster·cache good · Quadratic=secondary · DoubleHash=secondary없음 ($h_2 \neq 0 \cdot gcd=1$) · Rehashing=amortized O(1)·임계0.5~0.75 · Worst=O(n)

요약 Dictionary=insert·find·delete · hash table만 평균 O(1)(단 **adversarial이면 O(n)**) · division은 큰 소수 M · chaining(외부·α>1 허용·delete 간단) vs open addressing(내부·α<1·tombstone) · linear=primary cluster·quadratic=secondary·**double=secondary 없음** · rehashing amortized O(1)·α 0.5~0.75.

Ch 13 Graph Representations — 그래프 표현

인접 행렬 vs. 인접 리스트 · 공간/시간 trade-off · CSR/CSC

그래프 모델 & 용어 p.1-7

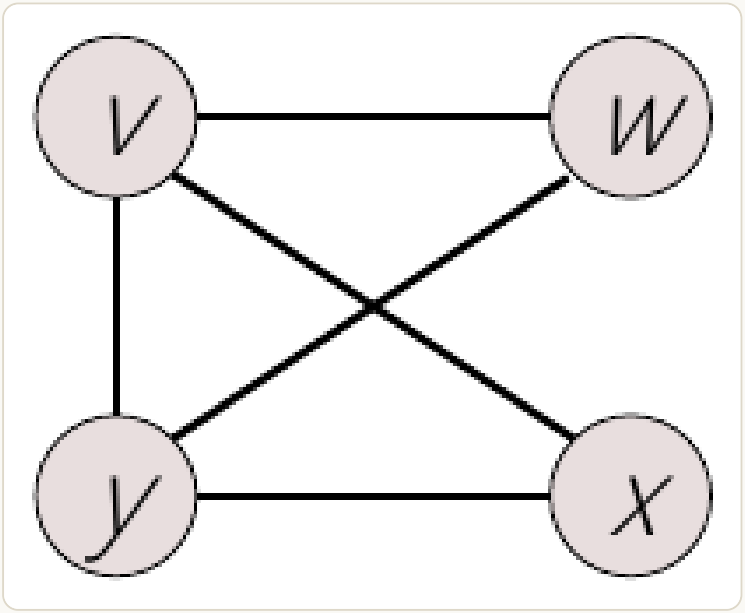
Q. adjacent vs incident 차이?

Q. complete·sparse·dense 기준은?

Q. subgraph vs induced subgraph?

Graph G = (V, E) — 정점 집합 V(|V|=n)와 간선 집합 E(|E|=e)의 순서쌍. 간선 (v,w)는 두 정점을 잇는 쌍.

- adjacent — 두 정점이 간선으로 직접 연결 (*정점-정점*). 방향 그래프: v is adjacent to w (v→w) / from v (←)
- incident — 간선이 정점에 닿아 있는 관계 (*정점-간선*). (v,w)는 v와 w에 incident



G = ({v,w,x,y}, {(v,w),(v,y),(w,y),(y,x),(x,v)}) 예제 (p.3)

간선 밀도에 따른 분류 — 간선 수 e가 n에 대해 어떻게 자라는가 — 자료구조 선택의 기준선.

- Complete graph K_n — 모든 정점 쌍 간선 존재. e = n(n-1)/2
- Sparse — `e = O(n)`. 소셜 네트워크·도로망 대부분
- Dense — `e = O(n²)`. 인접 행렬이 낭비 없는 구간

Path / Cycle / Simple graph — 경로 = 정점 시퀀스를 잇는 간선 시퀀스. *길이* = 간선 수.

- Simple path — 반복 정점 없는 경로
- Cycle — 시작=끝 정점인 경로 (길이 ≥ 3)
- Simple graph 가정 — 두 정점 사이 간선 최대 1개, self-loop 없음

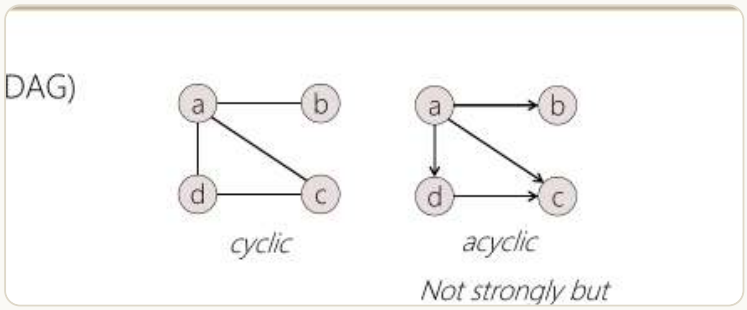
Subgraph vs. Induced subgraph — V_s를 고른 뒤 간선을 어떻게 결정하는가의 차이.

V_s 고정 → E_s 유일 결정

- Subgraph G_s — V_s ⊆ V, E_s ⊆ E 자유 선택 (단 양 끝점 ∈ V_s)
- Induced subgraph — V_s 지정 시 E_s가 자동 결정: E_s = {(v,w)∈E | v,w∈V_s}

Acyclic & Connectedness — 사이클 유무 + 연결성. 방향/무방향에 따라 세 갈래.

- DAG (Directed Acyclic Graph) — 방향 + 비순환. 위상 정렬 기반
- Connected (무방향) — 임의의 두 정점 사이 경로 존재 ↔ component = 1개
- Strongly connected (방향) — 모든 정점 쌍에 양방향 경로 존재
- Weakly connected (방향) — 방향 제거한 underlying 무방향 그래프가 connected



cyclic vs. acyclic(DAG) · weakly connected 예시 (p.7)

Tree · Spanning Tree · MST p.8-11

Q. tree의 동치 조건(연결&비순환)?
Q. spanning tree의 간선 수는?
Q. MST(최소비용 신장트리)란?

Connected Component — 연결된 *maximal subgraph*. G가 connected ↔ component가 정확히 1개.

- 방향 그래프에서는 반드시 strongly connected component로 명시

Tree — 다음 동치 조건 중 하나를 만족하는 *무방향 그래프*.

최소 연결 = 최대 비순환

- (a) connected & acyclic — 사이클 없는 연결 그래프
- (b) connected & 정점 n개에 간선 n-1개
- 간선 하나 추가 → cycle, 간선 하나 제거 → disconnected

Spanning Tree — 연결 그래프 G의 *모든 정점을 포함*하는 트리. V_T = V_G, |E_T| = n-1.

- G에 여분 간선(사이클)이 있으면 신장 트리는 유일하지 않다
- 가중치 부여 시: 간선 가중치 합이 최소인 신장 트리 = MST

MST (Minimum-Cost Spanning Tree) — 신장 트리들 중 *tree cost* (= 간선 가중치 합) 최소인 것. Kruskal/Prim(Ch.16)의 출발점.

- tree cost 비교: ST-1 cost=16, ST-2 cost=17 → ST-1이 MST

Adjacency Matrix — 인접 행렬 p.12-14

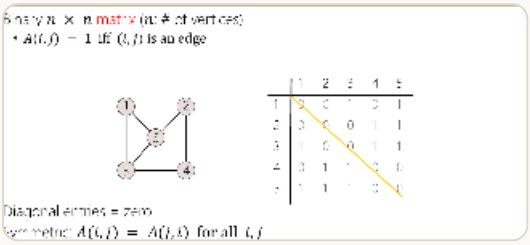
Q. 공간·간선조회·이웃순회 복잡도?

Q. 무방향 vs 방향 행렬 성질?

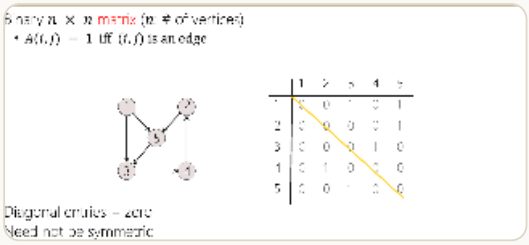
정의 — n×n *binary matrix*. A(i,j) = 1 iff (i,j) is an edge.

A(i,j)=1 iff edge (i,j)

- 공간: O(V²) — 정점 1만 개면 셀 1억 개. dense/소규모 그래프에 적합
- 간선 존재 여부 조회: O(1) — 행렬 인덱스 직접 접근
- 정점 v의 이웃 순회: Θ(n) — 행(또는 열) 전체를 봐야 함



무방향: A(i,j)=A(j,i) 대칭, 대각=0 (p.13)



방향: 비대칭 가능. 행합=out-deg, 열합=in-deg (p.14)

	무방향 Undirected	방향 Directed
대칭성	A(i,j) = A(j,i) 항상	비대칭 가능
대각선	0 (no self-loop)	0 (no self-loop)
행의 합	degree(i)	out-degree(i)
열의 합	degree(i)	in-degree(i)

인접 행렬 무방향 vs. 방향 핵심 성질

⚠ **함정** 방향 그래프에서 A(i,j)=1이라도 A(j,i)=0일 수 있다. 행합=out-degree, 열합=in-degree — 방향 그래프에서만 의미.

Sparse Matrix & CSR / CSC p.15

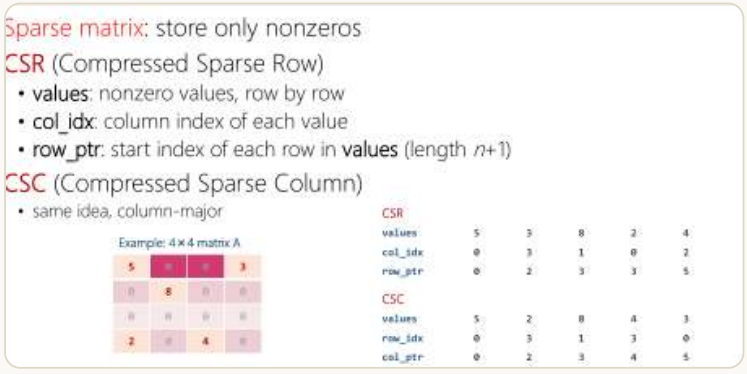
Q. CSR의 세 배열은?

Q. row_ptr 마지막 원소의 의미?

CSR (Compressed Sparse Row) — 인접 행렬이 sparse할 때 0을 저장하지 않고 nonzero만 압축. Array 인접 리스트와 *동일한 레이아웃*.

row_ptr 마지막 = 총 nonzero 수

- `values` — nonzero 값, 행 순서로 나열
- `col_idx` — 각 값의 열 인덱스 (values와 같은 길이)
- `row_ptr` — 각 행이 values에서 시작하는 인덱스 (길이 n+1)
- CSC — 같은 아이디어를 column-major로: values, row_idx, col_ptr



4×4 희소 행렬 → CSR/CSC 배열 예제 (p.15)

✎ **핵심** CSR row_ptr 해석: 행 i의 nonzero는 values[row_ptr[i] : row_ptr[i+1]]. 마지막 원소 row_ptr[n]=총 nonzero 수 (sentinel).

Adjacency Lists — 인접 리스트 p.16-19

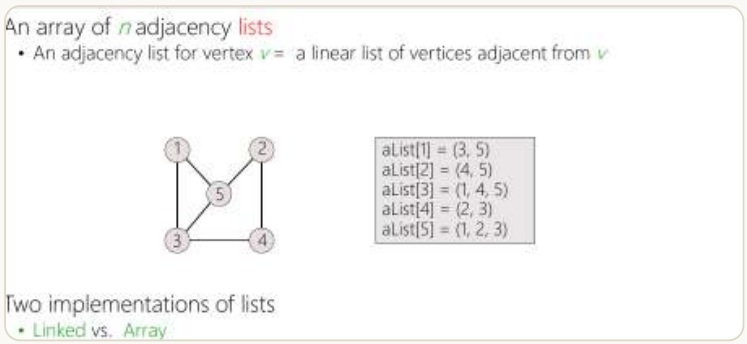
Q. 공간·이웃순회 복잡도?

Q. linked/single/multiple array 차이?

정의 — 정점마다 하나씩, 총 n개의 인접 리스트. v의 리스트 = *adjacent from v*인 정점들.

공간 O(n+e) vs. O(n²)

- 리스트 길이 = out-degree(v) (방향) / degree(v) (무방향)
- 공간: O(n + e) — sparse 그래프에서 인접 행렬 O(n²) 대비 압도적 절약
- 이웃 순회: O(deg(v)) — 리스트 길이만큼만



aList[1]=(3,5), aList[2]=(4,5), ⋯ 예제 (p.16)

구현 1 — Linked adjacency list — 각 인접 리스트가 *포인터로 연결된 chain*.

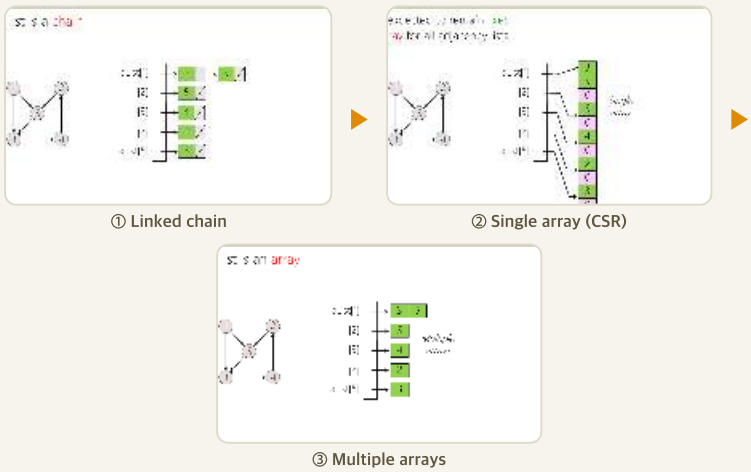
- 장점: 삽입/삭제 용이, 실제 간선 수만큼만 노드 생성
- 단점: 포인터 오버헤드 + cache locality 낮음 (노드 산발)
- 언제: 그래프가 *동적으로 변할 때*

구현 2 — Single array (= CSR) — 모든 인접 리스트를 하나의 연속 배열에 저장. 그래프 고정 가정.

- 장점: 캐시 친화적, 포인터 없음 → 순회 빠름
- 단점: 중간 삽입 시 재배치 필요 — 동적 그래프에 부적합
- 언제: 빌드 후 고정, 대량 순회

구현 3 — Multiple arrays — 정점마다 *독립적인 작은 배열*. Single array와 Linked의 중간.

- Linked보다 접근 빠름(포인터 추종 없음), 길이 추적 쉬움
- Single array보다 연속성 낮음 — 리스트 간 캐시 효율 감소



① Linked chain ② Single array (CSR)

③ Multiple arrays

같은 directed 그래프의 세 가지 인접 리스트 구현 (p.17-19)

표현법 비교 & 선택 기준 p.20 Summary

Q. 행렬 vs 리스트 언제 선택?

Q. sparse 판단 기준 수치는?

	인접 행렬	인접 리스트
공간	$O(V^2)$	$O(n + e)$
간선 존재 조회	$O(1)$	$O(deg(v))$
이웃 순회	$\Theta(n)$ 행 전체	$O(deg(v))$
적합	Dense / 조회 빈번	Sparse / 순회 빈번
구현 단순성	단순 (2D 배열)	약간 복잡 (리스트 관리)

같은 그래프, 다른 표현 — Dense면 행렬, Sparse면 리스트

✎ **핵심** sparse 기준: $n=10^4$, $e=10^5$이면 인접 리스트 ≈ 1.1×10^5 셀, 인접 행렬 = 10^8 셀 → 약 *천 배* 차이. 실전(소셜·웹·도로망)은 거의 sparse → 인접 리스트가 기본값.

★ **필수암기** G=(V,E) · adjacent=정점↔정점, incident=정점↔간선 · Complete $e=n(n-1)/2$, Sparse $e=O(n)$, Dense $e=\Theta(n^2)$ · Tree: connected&acyclic, $|E|=n-1$ · 인접행렬: $O(V^2)$공간·$O(1)$조회·대칭(무방향)·행합=out-deg · 인접리스트: $O(n+e)$공간·$O(deg)$조회 · CSR=single-array=row_ptr/col_idx/values

요약 그래프 표현 = 인접행렬($O(V^2)$공간·$O(1)$ 간선조회·대칭(무방향)·행합=out-degree) vs 인접리스트($O(n+e)$공간·$O(deg)$ 순회·sparse 유리) · Tree=connected&acyclic·$|E|=n-1$ · CSR=row_ptr/col_idx/values · 밀도(dense/sparse)로 표현 선택.

Ch 14 Graph Traversals — 그래프 순회

mark bit로 안전성 확보 · 스택(DFS) vs 큐(BFS) 선택이 순회 성격 결정

Graph Traversal 개요 p.2-4

Q. traversal(체계적 1회 방문)이란?

Q. mark bit가 필요한 이유는?

Q. 대표 두 알고리즘은?

Graph Traversal / Search — 그래프의 모든 정점을 체계적으로 한 번씩 방문. 트리 순회의 일반화.

- 두 까다로운 경우: not connected (일부 정점 미도달) / cycle (무한 루프)
- 해결책 → mark bit (visited flag)를 정점마다 유지: 이미 방문한 정점 재진입 차단
- 트리 순회는 acyclic + unique path → mark 불필요; 그래프는 두 속성 모두 없음 → *필수*
- 대표 알고리즘: DFS (재귀 스택) / BFS (명시적 큐)

Depth-First Search (DFS) p.5-13

Q. DFS 정의(가능한 깊이까지)?

Q. 재귀/호출 스택 구현은?

Q. backtrack 시점은?

DFS 정의 — 각 자식을 가능한 한 깊게 탐색 후 backtrack. 트리 preorder traversal의 그래프 버전.

- 구현: *재귀(recursion stack)* 기반 — 호출 스택이 현재 경로를 유지
- backtrack = unvisited 인접이 없으면 호출 스택을 거슬러 올라가 가장 최근의 unvisited 형제로

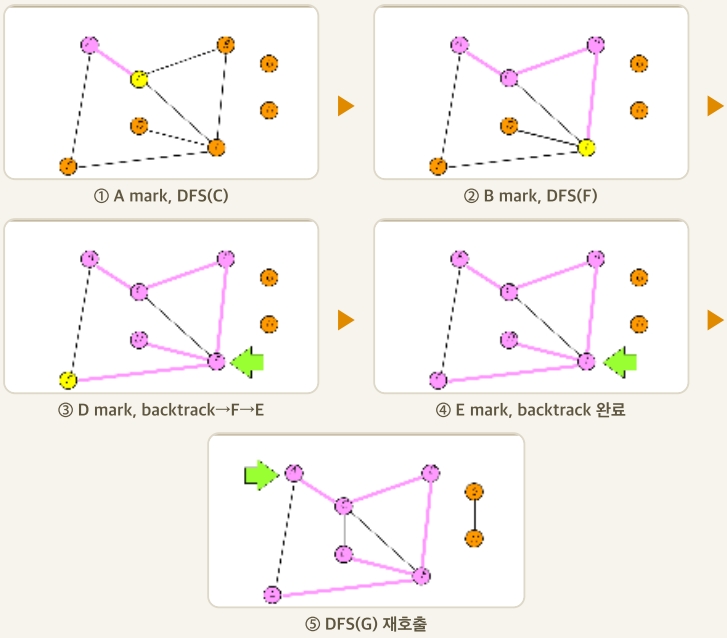
DFS(v)

```
mark[v] := visited;
for (each unvisited vertex w adjacent from v)
  DFS(w);
```

mark 후 unvisited 인접을 재귀 — 두 줄이 DFS의 전부.

DFS 전체 과정 예제 (A~H) — A에서 출발. A→C→B→F→D backtrack→E backtrack→A 완료. G·H는 별도 호출.

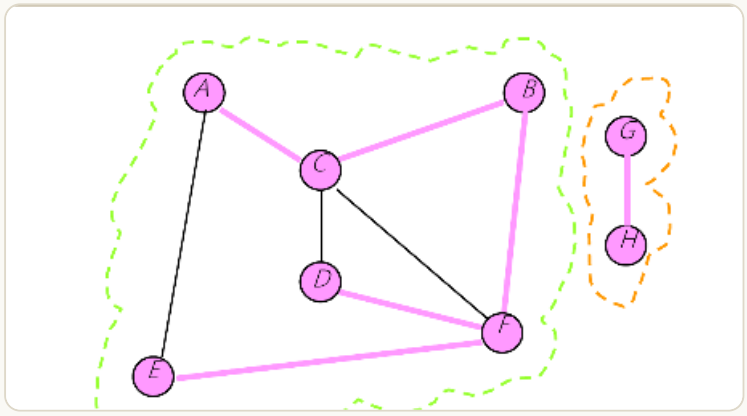
- 호출 스택 절정: `DFS(A)→DFS(C)→DFS(B)→DFS(F)→DFS(D)`
- G, H는 A의 DFS 완료 후 *새 DFS(G) 호출* → spanning forest의 두 번째 트리



DFS 전체 과정 — mark·재귀·backtrack 흐름 (p.6-12)

Depth-First Spanning Forest — pink edges = tree edge 집합 → spanning forest. 연결 성분 하나 = 트리 한 그루.

- tree edge 수 = 전체 정점 수 − DFS tree 수 (비순환 + 전정점 포함 = 트리 정의)
- forest의 트리 개수 = connected component 개수



DFS spanning forest — 2개 성분(녹색·주황 점선) (p.13)

DFS 시간 복잡도 & 응용 p.14-16

Q. DFS 복잡도(리스트/행렬)?

Q. DFS 한 번으로 얻는 정보는?

시간 복잡도 — 모든 정점은 정확히 *한 번*만 방문. 인접 탐색 비용이 자료구조에 따라 다름.

리스트: O(n+e)
행렬: O(n²)

- 인접 리스트: 정점 방문 시 O(degree) → aggregate: T = c·n + e → O(n+e)
- 인접 행렬: 정점 방문 시 O(n) → T = n·(c_1+c_2·n) → O(n²)
- sparse 그래프에서는 인접 리스트가 압도적으로 유리

DFS 응용 — DFS 한 번으로 여러 구조적 정보를 동시에 얻는다.

- Spanning Tree: unvisited 정점 도달 시 사용한 간선들 → G가 connected이면 n−1개 간선
- Path Finding: v에서 DFS 시작, w 방문되거나 unvisited 소진 시 종료
- Connected 판별: 임의 정점 DFS 후 모든 정점 방문됐으면 connected
- Connected Components: unvisited 정점마다 DFS 재시작 → 각 호출 = 한 component

Strong Components — Kosaraju 알고리즘 p.17-20

Q. strong component 정의는?

Q. Kosaraju 3단계는?

Q. 완료 순서(postorder)가 왜 중요?

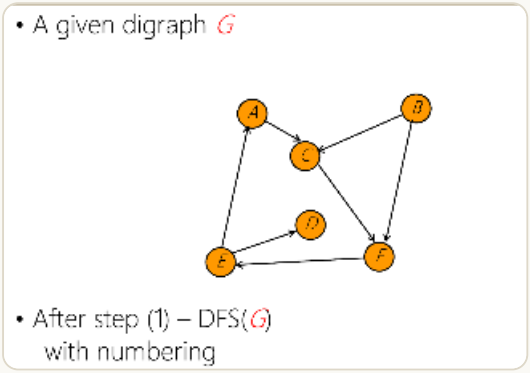
Strong Component — 유향 그래프에서 **서로 도달 가능한** 정점의 최대 집합.

- 입력: digraph G / 출력: strong components 집합

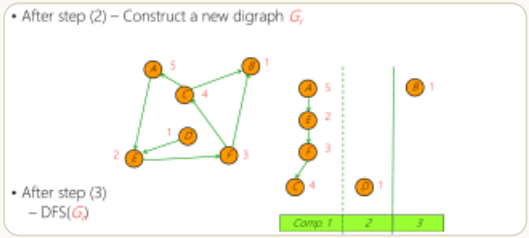
Kosaraju — Strong Components

① **DFS(G):** 재귀 완료 순서로 각 정점에 번호 부여
(완료 순서 = **postorder** — 방문 순서와 다름!)
② **G**의 모든 **arc** 방향 뒤집기 → **Gr** 생성
③ **DFS(Gr):** ①의 번호가 가장 큰 정점부터 시작
→ 각 **DFS** 호출이 도달하는 정점들 = 하나의 **strong component**

복잡도: 두 번의 DFS = O(n+e). 완료 순서가 sink를 먼저 가리키게 만드는 것이 핵심.



Step①: G의 DFS — 완료번호 부여 (p.18)



Step②③: Gr + DFS → 3개 성분 (p.19)

⚠ **함정** 완료 순서(*postorder*)와 방문 순서를 혼동하지 말 것. 완료 번호가 **큰** 정점 = DFS에서 *가장 늦게 완료* = sink 쪽. Gr에서는 이 정점에서 출발해야 source 쪽을 묶을 수 있다.

Topological Sort — 위상 정렬 p.21-24

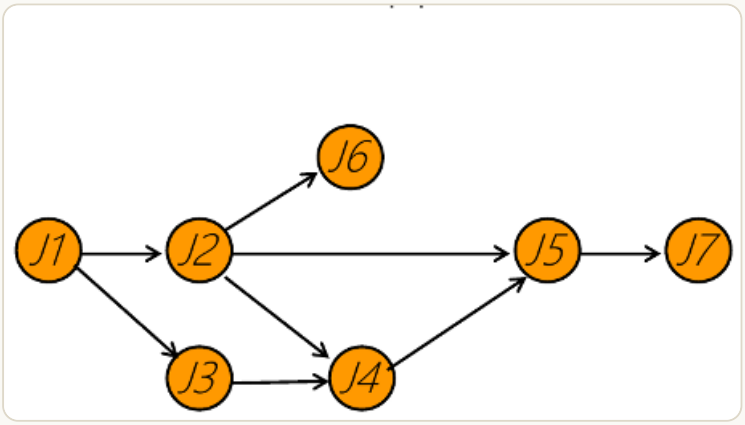
Q. topo sort 정의와 전제(DAG)?

Q. DFS 기반 방법(완료시점 역순)?

Q. 큐(in-degree) 기반·cycle 검출?

Topological Sort — DAG 정점의 **선형 순서**: path v→w가 있으면 v가 w보다 앞에 온다.

- 전제 조건: **DAG (Directed Acyclic Graph)** 에만 정의됨
- valid ordering은 일반적으로 여러 개 존재 (unique하지 않음)



예제 DAG: J1→J2→J5→J7 등 다수의 valid ordering 존재 (p.21)

방법 1 — DFS 기반 (Stack) — DFS 수행 시 정점 *완료 시점*에 print → 결과는 topo sort의 **역순**.

- 방문 시점: 아무 동작 없음 / 완료(pop back) 시점: print
- 역순을 뒤집으면 유효한 topological ordering

TOPSORT(v) — DFS 기반

```
mark[v] := visited;
for (each unvisited vertex w adjacent from v)
  TOPSORT(w);
print(v);    ← 완료 시점에 출력 (역순 → 뒤집으면 topo order)
```

방법 2 — Queue 기반 (in-degree) — in-degree 0인 정점을 enqueue → dequeue & 출력 → 나가는 간선 제거 → 반복. 결과가 정방향.

큐: 정방향
DFS: 역순

- in-degree 0 정점이 여럿이면 어느 것을 먼저 골라도 valid ordering
- Cycle 검출: 처리 후 남은 정점 수 > 0 이면 → cycle 존재 판정

방식	자료구조	출력 순서	cycle 검출
DFS 기반	recursion stack	역순 → 뒤집기	별도 처리 필요
큐(in-degree)	FIFO queue	정방향	처리 후 미처리 정점 > 0

두 방식 비교. 큐 기반이 cycle 검출에도 직접 활용 가능.

Breadth-First Search (BFS) p.25-34

Q. BFS 정의(가능한 넓이까지)?

Q. 자료구조(FIFO 큐)는?

Q. unweighted 최단경로 보장 이유?

BFS 정의 — 같은 depth의 정점을 가능한 한 넓게 모두 방문 후 다음 depth로. level-order 탐색.

- 구현: *iterative* — DFS 재귀 스택 대신 FIFO queue 사용
- 스택 오버플로 없음, 깊이 제한 없음

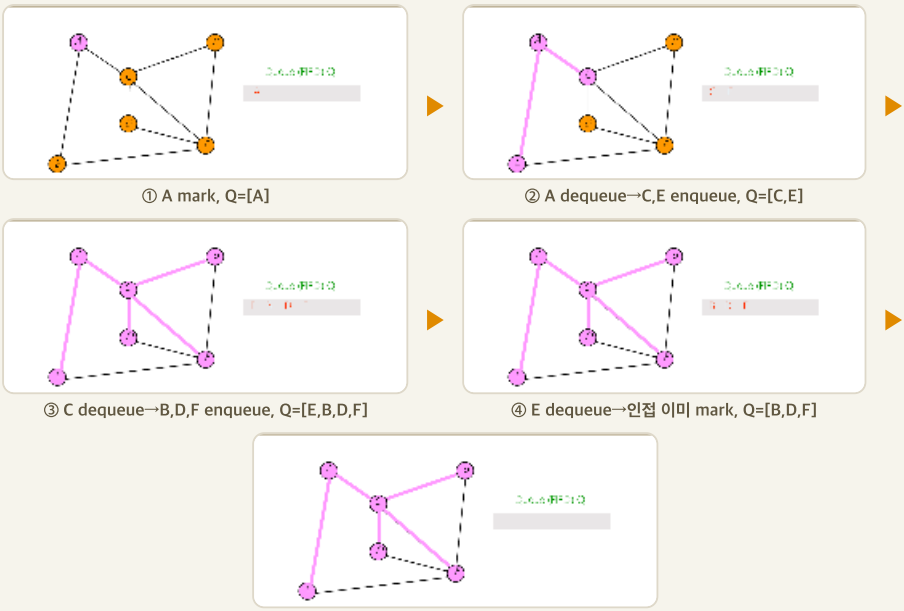
BFS

```
Visit/Mark start vertex & enqueue;
repeat until queue is empty:
  dequeue vertex v;
  for (each unvisited vertex w adjacent from v):
    Visit/Mark w;
    enqueue w;
```

dequeue 순서 = 방문 순서 = 거리 순서(source로부터 간선 수 기준 오름차순).

BFS 전체 과정 예제 — Q=[A] → A dequeue, C·E enqueue → C dequeue, B·D·F enqueue → E·B·D·F 처리 → Q empty.

- 방문 순서: A → C, E (depth 1) → B, D, F (depth 2)
- 처음 mark되는 시점의 depth = source로부터의 최단 거리(간선 수)



① A mark, Q=[A]　② A dequeue→C,E enqueue, Q=[C,E]

③ C dequeue→B,D,F enqueue, Q=[E,B,D,F]　④ E dequeue→인접 이미 mark, Q=[B,D,F]

⑤ Q empty → 종료

BFS 전체 과정 — FIFO가 곧 level-order 보장 (p.26-33)

BFS 성질 & 응용 — DFS와 동일한 O(n+e) 복잡도(인접 리스트). 동일하게 적용 가능한 성질:

- Connectivity 판별 / Connected components 식별 / Spanning tree / Path finding
- unweighted 최단 경로: BFS가 유일한 표준. level-order = 거리 순서 → 처음 도달 = 최단
- DFS는 첫 경로가 최단이라는 보장 없음 → unweighted 최단경로에 *부적합*

DFS vs BFS 비교 p.35-36

Q. 자료구조·탐색방향 차이?

Q. 최단경로는 어느 쪽 우위?

	DFS	BFS
자료구조	recursion stack (암시적)	**explicit FIFO queue**
탐색 방향	깊이 우선 (depth-first)	너비 우선 (breadth-first)
시간복잡도	**O(n+e) 리스트 / O(n²) 행렬**	**O(n+e) 리스트 / O(n²) 행렬**
spanning tree	DFS spanning forest	BFS spanning tree
unweighted 최단경로	**불가 (경로 비최단 가능)**	**BFS 우위 (level=거리)**
응용 특화	strong components, topo sort	최단경로(unweighted)

두 알고리즘의 자료구조 선택 하나가 탐색 성격과 응용 범위를 결정.

✎ **핵심** mark bit + stack(DFS) = 깊이 우선·재귀·backtrack·spanning forest·strong components·topo sort. mark bit + queue(BFS) = 너비 우선·iterative·level-order·unweighted 최단경로. 복잡도는 인접 리스트 기준 둘 다 O(n+e) — 인접 행렬이면 O(n²).

★ **필수암기** DFS=재귀스택·O(n+e)/O(n²)·spanning forest·component수=트리수·strong comp(Kosaraju 2×DFS+Gr)·topo sort(완료시점print역순) · BFS=FIFO큐·동일복잡도·level-order·처음mark=최단거리(unweighted) · cycle→topo sort 불가·큐기반 미처리정점>0이면cycle

요약 Graph traversal = **mark bit**로 재방문 차단 · DFS(재귀스택·spanning forest·strong comp=Kosaraju 2×DFS+G_r·topo sort) · BFS(FIFO 큐·level-order·**unweighted 최단경로**) · 둘 다 인접리스트 O(n+e)·행렬 O(n²) · cycle이면 topo sort 불가.

Ch 15 Shortest Path — 최단 경로

가중 유향 그래프에서 비용 합이 최소인 경로 찾기 · Dijkstra · Floyd

문제 정의 & 4가지 유형 p.1-8

Q. SPP 4가지 유형과 대표 알고리즘?

Q. path length(경로 길이)의 정의는?

Q. naive greedy가 실패하는 이유는?

SPP (Shortest Path Problem) — *비음(non-negative)* 가중 유향 그래프에서 두 정점 사이 **간선 가중치 합이 최소**인 경로를 찾는 문제. path length = 경로 위 모든 가중치의 합.

- **Single-pair** — source 1개 · destination 1개
- **Single-source** — source 1개 · **모든** destination ⟶ **Dijkstra**
- **Single-destination** — 간선 뒤집으면 single-source로 환원
- **All-pairs** — 모든 쌍 ⟶ **Floyd**

Naive Greedy의 실패 — "도달 가능한 정점 중 *최저 가중 간선*으로 다음 정점 선택"하는 단순 greedy는 전역 최적을 보장 못 함.

- A→E 예시: greedy는 길이 **21**을 내지만 실제 최단은 **18** ⟶ *Algorithm doesn't work*
- Dijkstra는 같은 greedy지만 **D[v]의 정의**가 달라서 동작 (아래)

Dijkstra Algorithm single-source · 비음 가중치

Q. D[i]·P[i]는 각각 무엇을 뜻하나?

Q. 매 stage에서 어떤 정점을 S에 넣나?

Q. 정당성을 귀류법으로 어떻게 보이나?

Q. 음수 간선이면 왜 무너지나?

Q. $O(n^2)$ vs $O(e \log n)$은 언제 유리?

상태 두 배열 `D[i]` · `P[i]` — `D[i]` = source에서 i까지의 *현재* 최단거리 (= **S 안의 정점만 통과**하는 최단경로 길이). `P[i]` = i 직전 predecessor → 역추적으로 실제 경로 복원.

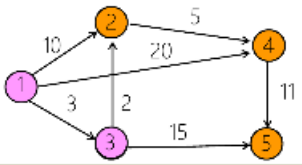
D=거리, P=직전점
S=확정집합

- `S` = source로부터 최단거리가 *이미 확정된* 정점 집합
- 매 stage: `V−S` 중 **D[.] 최소**인 `w` 를 S에 추가 → 그 순간 D[w]가 곧 최단거리

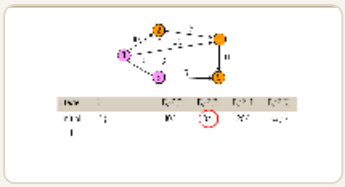
Dijkstra (경로까지 기록)

```
Initialize:
  S := {1}
  for i := 2 to n do
    D[i] := C[1,i]
    if C[1,i] ≠ ∞ then P[i] := 1     ⟵ source를 직전점으로
for stage := 1 to n−1 do
  w := argmin D[v] for v in V−S      ⟵ one-edge extension
  S := S ∪ {w}
  for each v in V−S do               ⟵ relaxation
    if D[w] + C[w,v] < D[v] then
      D[v] := D[w] + C[w,v]
      P[v] := w
```

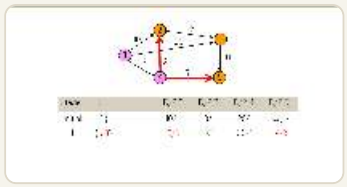
relaxation = 한-간선 이완. D[w]+C[w,v]가 기존 D[v]보다 작으면 갱신.



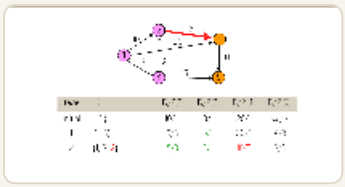
예제 그래프 — source=1 (p.13)



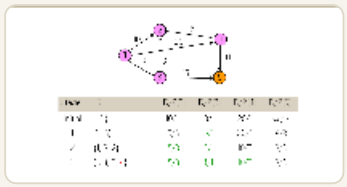
① Initial — S={1}



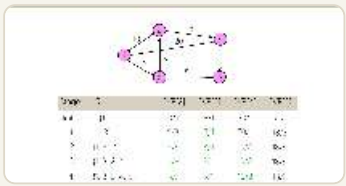
② Stage 1 — 3 확정



③ Stage 2 — 2 확정



④ Stage 3 — 4 확정



⑤ Stage 4 — 5 확정

Dijkstra 전체 진행 — 빨간 화살표=relax된 간선, 녹색=확정 (p.13-17)

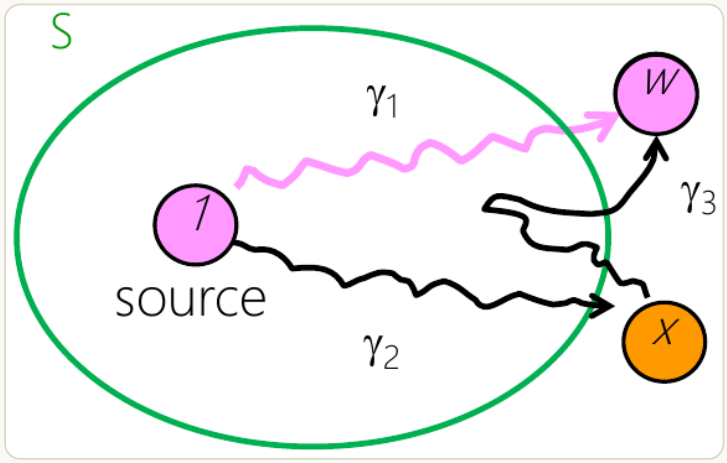
Progress 추적 — 전체 과정 — stage마다 D[.] 최소 정점을 확정(S에 추가)하고 이웃을 relax. 거리/직전점을 D/P로 표기.

Stage	S	D/P[2]	D/P[3]	D/P[4]	D/P[5]
Initial	{1}	10/1	3/1	20/1	∞ / -
1	{1,3}	5/3	3/1	20/1	18/3
2	{1,3,2}	5/3	3/1	10/2	18/3
3	{1,3,2,4}	5/3	3/1	10/2	18/3
4	{1,3,2,4,5}	5/3	3/1	10/2	18/3

확정(초록)·relax로 갱신된 값(분홍). 최단거리 D[5]=18은 1→3→5(3+15).

정당성 — Greedy Choice Property — 매 단계 *locally-best*(D 최소) 선택이 *전역 최선*임을 **귀류법**으로 증명.

- 가정: S 밖 정점 x 경유 경로 $|\gamma_2|+|\gamma_3| < |\gamma_1|$(현재 D[w])
- 그러면 $|\gamma_2| < |\gamma_1|$ → **x가 w보다 먼저 뽑혔어야 함** → **모순**



S 밖 x 경유가 더 짧다면 모순 (p.21)

⚠ **함정** 이 증명은 **비음 가중치** 가정에 의존. **음수 간선**이면 x→w가 음수일 때 $|\gamma_2|+|\gamma_3|<|\gamma_1|$이어도 $|\gamma_2|<|\gamma_1|$이 깨짐 ⟶ Dijkstra가 *틀린 최단거리*를 보고할 수 있다.

복잡도 — 자료구조에 따라 둘 — 병목은 "*D 최소 정점 선택*" 연산.

- Option-1 전체 리스트 스캔: 선택 $O(n^2)$ + 갱신 $O(e)$ = $O(n^2)$ → *밀집 그래프* 유리
- Option-2 min-heap: DeleteMin n번 $O(n \log n)$ + 갱신 e번 $O(e \log n)$ = $O(e \log n)$ → *희소 그래프* 유리

Min-Heap DeleteMin

Option-2의 핵심 연산 · O(log n)

Q. DeleteMin의 2단계는?

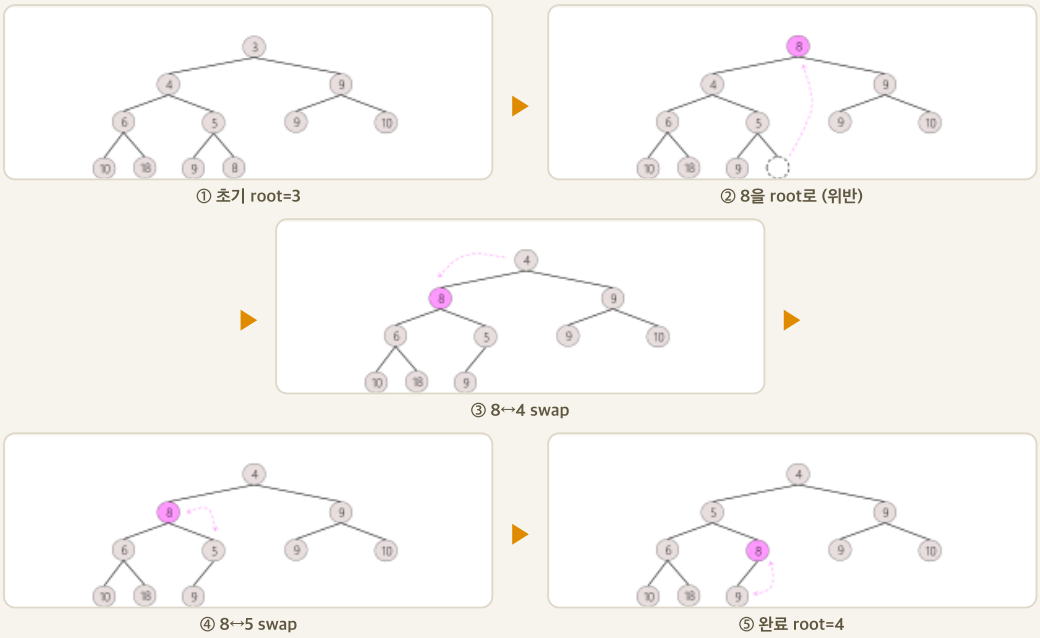
Q. down-heap에서 어느 자식과 비교? 왜?

Q. 왜 비용이 O(log n)인가?

Q. Dijkstra는 사이클·DAG에서?

DeleteMin — min-heap(부모≤자식, 완전이진트리)에서 최소(root)를 빼서 반환. 2단계로 동작.

- Step-1 root 제거 → *마지막 leaf*를 root 자리(hole)로 이동
- Step-2 down-heap bubbling — 더 작은 자식과 비교·swap을 leaf까지 반복



down-heap bubbling 전체 과정 — 더 작은 자식과 swap 반복 (p.25-33)

✎ 핵심 down-heap에서 두 자식 중 더 작은 값과 비교해야 swap 후에도 부모≤두자식이 유지된다. 비용 ∝ height = O(log n) → Dijkstra heap 버전 O((n+e)log n)이 여기서 나옴.

Dijkstra 확장·한계

- Cyclic graph — OK. update rule이 우회경로를 즉시 갱신
- Negative edge — 실패. greedy choice 증명이 무너짐
- DAG — topological sort 순서로 relaxation 한 패스면 충분(더 빠름)

Floyd Algorithm

all-pairs · DP · $O(n^3)$

Q. $A_k[i,j]$의 정의는?

Q. Floyd 점화식을 써보면?

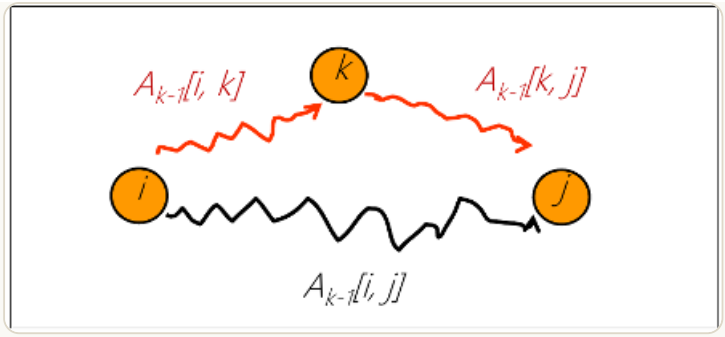
Q. 시간·공간 복잡도와 in-place 근거?

Q. Next 테이블로 경로를 어떻게 복원?

DP 정의 $A_k[i,j]$ — i→j 경로 중 중간 정점 인덱스가 모두 ≤ k인 것의 최단 길이. k를 0→n까지 키우며 k번째 정점을 차례로 *중간 경유 허용*.

중간점 ≤ k 허용하며 확장

- 점화식: $A_k[i,j] = \min\{ A_{k-1}[i,j],\ A_{k-1}[i,k]+A_{k-1}[k,j] \}$
- k 경유 vs 직접을 비교 — 모든 (i,j) 쌍에 적용



k 경유 vs 직접 (p.39)

Floyd + 경로기록(Next)

```
for k := 1 to n do
  for i := 1 to n do
    for j := 1 to n do
      if A[i][k] + A[k][j] < A[i][j] then
        A[i][j]    := A[i][k] + A[k][j]
        Next[i][j] := Next[i][k]        ← 경로 successor 갱신
```

Next[u][v] 초기값: 간선 있으면 v, 없으면 -1. in-place 갱신 가능(공간 $O(n^2)$).

✎ **핵심** Space reduction: A_k가 A_{k-1}을 덮어써도 안전(i=k,j=k 행·열은 불변, 나머지는 A_k 계산에만 쓰임) ⟶ $O(n^2)$ 공간, 시간은 여전히 $O(n^3)$.

행렬 진화 A^0 → A^5 (전체 과정) — 초기 A^0(직접 간선) → 중간점을 하나씩 허용하며 갱신 → 최종 A^5(모든 중간점 허용 = 정답).

A^0	1	2	3	4	5
1	0	10	3	20	–
2	–	0	–	5	–
3	–	2	0	–	15
4	–	–	–	0	11
5	–	–	–	–	0

A^0 — 중간점 없음(직접 간선)

A^5	1	2	3	4	5
1	0	5	3	10	18
2	–	0	–	5	16
3	–	2	0	7	15
4	–	–	–	0	11
5	–	–	–	–	0

A^5 — 최종 최단거리(정답)

경로 복원 — Next chain — 거리 행렬만으론 실제 경로 모름 → `Next` 테이블을 따라 hop.

Path retrieval (1→4)

```
path := [u]
while u ≠ v do
  u := Next[u][v]
  path.append(u)
u=1: Next[1,4]=3 → [1,3]
u=3: Next[3,4]=2 → [1,3,2]
u=2: Next[2,4]=4 → [1,3,2,4]  ✓ 거리 3+2+5=10 = A⁵[1,4]
```

비교 · 다른 알고리즘 p.51-55

Q. Floyd vs Dijkstra×n — 밀도별 승자?

Q. 음수 간선에서 동작하는 쪽은?

Q. Bellman-Ford·Johnson·A*의 용도는?

	Dijkstra ×n	Floyd
용도	single-source 반복	all-pairs 한 번
시간	$O(n^2 \log n)$ (heap, sparse)	$O(n^3)$
유리	희소(sparse) 그래프	밀집(dense) 그래프
음수 간선	불가(정당성 붕괴)	가능 (negative cycle만 없으면)

밀집·음수 간선이면 Floyd / 희소면 Dijkstra×n

기타 최단경로 알고리즘

- Bellman-Ford — single-source, 음수 간선 허용, $O(n \cdot e)$ (Dijkstra보다 느림)
- Johnson — all-pairs, 음수 허용, $O(n^2 \log n + n \cdot e)$ (희소면 Floyd보다↑)
- A* search — single-pair, *휴리스틱*으로 탐색을 좁은 cone으로 가속

> ★ **필수암기** Dijkstra=single-source·비음·$O(n^2)$/$O(e \log n)$ · D[v]=S내부만통과 최단 · 음수간선 실패 · Floyd=all-pairs·$O(n^3)$·DP점화식·음수가능(neg-cycle 없을때) · DeleteMin=O(log n)

요약 최단경로 = Dijkstra(single-source·**비음 가중치**·greedy+귀류법·$O(n^2)$ 또는 $O(e \log n)$, D[v]=S 내부만 통과) · Floyd(all-pairs·DP 점화식·$O(n^3)$·음수 간선 OK, negative cycle만 없으면) · DeleteMin=O(log n).

Ch 16 Minimum Spanning Trees — 최소 비용 신장 트리

가중 연결 무방향 그래프에서 모든 정점을 연결하는 최소 비용 트리 · Prim · Kruskal · Sollin

MST 정의 & Greedy 전략 3가지 p.1-8

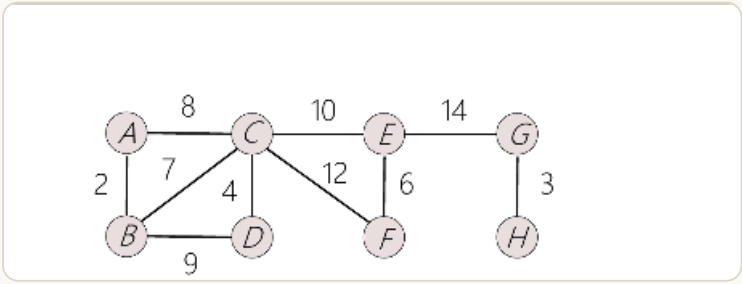
Q. MST(스패닝 트리 중 최소)란?

Q. Prim·Kruskal·Sollin 차이는?

Q. 순진한 greedy가 실패하는 이유?

MST (Minimum-Cost Spanning Tree) — 가중 *연결 무방향* 그래프에서 모든 정점을 포함하면서 간선 비용의 합이 최소인 트리.

- spanning tree는 항상 n-1개 간선 → n개 간선 중 n-1개 선택(또는 나머지 버리기) 문제
- 입력 조건: *가중 · 연결 · 무방향* — 비연결이면 spanning tree 자체 불가
- 응용: 네트워크 인프라 · 도로망 · 회로 배선 ("모두 연결 + 총비용 최소")



예제 그래프 — 8 정점 10 간선, spanning tree = 7개 간선 (p.5)

세 가지 Greedy 전략 — 모두 cut property 기반 greedy. 차이는 '어떤 단위로 간선을 고르느냐'.

- Prim — *1-정점 트리*에서 시작, 매 단계 가장 싼 간선(+정점)을 붙여 n-정점 트리로 성장
- Kruskal — *n-정점 forest*에서 시작, 간선을 비용 오름차순으로 검토, 사이클 안 만들면 채택
- Sollin (Borůvka) — 각 component가 *동시에* 최소 inter-cluster 간선 선택 → 병렬화 강점

> ⚠ **함정** "각 정점의 가장 싼 outgoing edge"를 단순히 고르는 *순진한 greedy*는 동작하지 않는다 — 사이클 발생, 전역 최적 미보장. Prim·Kruskal은 cut property로 '안전한 선택'임이 보장된다.

Prim 알고리즘 p.9-17

Q. 3대 자료구조 Q·D[.]·P[.]는?

Q. 매 step 무엇을 트리에 추가?

Q. decrease-key란?

3대 자료구조 `Q` , `D[k]` , `P[k]` — 우선순위 큐 기반 구현의 핵심 세 배열.

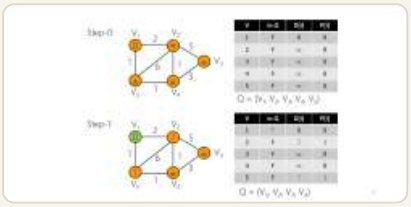
D=트리거리
P=부모
Q=미편입큐

- Q — 아직 트리에 포함되지 않은 정점들의 우선순위 큐
- D[k] — 정점 k의 현재 트리까지 최단 거리 (= Q 안의 우선순위)
- P[k] — k를 트리에 잇는 부모 정점 (MST 간선 복원용)

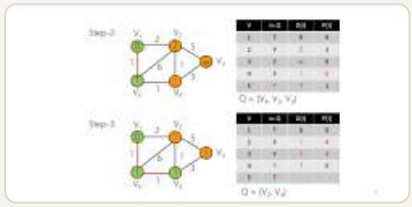
Prim (우선순위 큐)

```
Initialize:
  Q ← V
  D[1] := 0;  P[1] := 0
  for i := 2 to n do  D[i] := ∞, P[i] := 0
while Q ≠ ∅ do
  w ← DeleteMin(Q)            ← D[.] 최소 정점 트리에 편입
  for each v in Adj[w] do
    if v ∈ Q and C[w,v] < D[v] then
      D[v] := C[w, v]         ← decrease-key
      P[v] := w
```

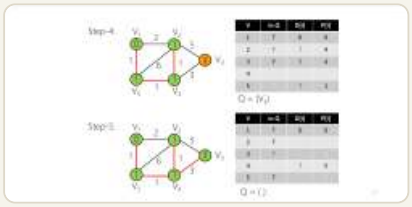
decrease-key: 새 트리 정점 w 덕에 바깥 정점의 D[v]가 더 싸질 때 갱신.



Step-0~1: $V_1 \rightarrow V_5$(D=1)



Step-2~3: $V_5 \rightarrow V_4$(D=1)·decrease-key V_2



Step-4~5: $V_2 \rightarrow V_3$ 완료

Prim step-by-step — 초록=확정, 빨강=MST 간선, 오렌지=현재 큐 최솟값 (p.15-17)

D/P 전체 추적 — step마다 DeleteMin 직후 D·P 상태. 초기 D[1]=0, 나머지 ∞.

단계	D[.] 갱신 (decrease-key)	Q 상태
Init	V_1=0,V_2=∞,V_3=∞,V_4=∞,V_5=∞	모두 0
Step-1 (V_1↑)	D[V_2]:=2(P=1), D[V_5]:=1(P=1)	Q=(V_5,V_2,V_3,V_4)
Step-2 (V_5↑)	D[V_4]:=1(P=5)	Q=(V_4,V_2,V_3)
Step-3 (V_4↑)	D[V_2]:=1(P=4)↓, D[V_3]:=3(P=4)	Q=(V_2,V_3)
Step-4 (V_2↑)	갱신 없음	Q=(V_3)
Step-5 (V_3↑)	MST 완성	Q=∅

분홍=decrease-key 발생, 초록=완성. MST 간선: V_1-V_5(1)·V_5-V_4(1)·V_4-V_2(1)·V_4-V_3(3)

Prim 정당성 & 복잡도 p.18-23

Q. cut property(MST property)란?

Q. greedy-choice 귀류법 골격은?

Q. 복잡도(스캔 vs min-heap)?

MST Property (Cut Property) — 연결 그래프 G=(V,E)와 진부분집합 U⊂V에 대해, u∈U·v∈V-U를 잇는 최소 비용 간선 (u,v)를 포함하는 MST가 반드시 존재한다.

- 이것이 greedy-choice property — 국소 최적 선택이 전역 최적으로 이어짐
- Prim은 매 step 'U=트리 / V-U=바깥'이라는 cut에 이 성질을 반복 적용

귀류법 증명 — (u,v)를 포함하는 MST가 없다고 가정 → T에 (u,v) 추가 시 사이클 생성 → 사이클 안의 다른 cut 간선 (u',v') 존재

- 가정상 c(u,v) ≤ c(u',v') → (u',v')를 (u,v)로 교체 → 비용 같거나 감소한 또 다른 MST 모순
- 따라서 (u,v)를 포함하는 MST 존재 — cut property 성립
- 음수 간선도 OK: MST는 최단 경로가 아니라 총비용 최소만 따짐 → 교환 논증에 영향 없음

복잡도 — 자료구조에 따라 둘 — 병목은 '*D 최소 정점 선택*(DeleteMin) 연산.

- Adjacency matrix + 선형 스캔: 초기화 O(n) + while n번 × 스캔 O(n) = O(n²) → *dense* 그래프 유리
- Adjacency list + min-heap: DeleteMin n번 O(n log n) + decrease-key e번 O(e log n) = O(e log n) → *sparse* 그래프 유리

Kruskal 알고리즘 p.24-31

Q. 간선을 어떤 순서로 검토?

Q. 사이클 판정은 어떻게?

Q. 복잡도와 병목은?

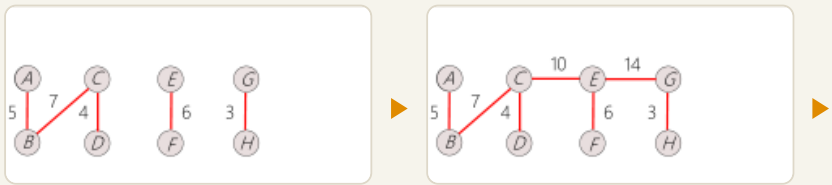
Kruskal — 간선 중심 Greedy — 간선 없는 forest(정점 n개)에서 시작. 간선을 비용 오름차순으로 검토 — 사이클 안 만들면 채택, 만들면 버린다.

- n-1개 간선이 모이면 MST 완성. '사이클 판정' = Union-Find의 FIND 결과 비교
- 채택 순서: (G,H)=3 · (C,D)=4 · (A,B)=5 · (E,F)=6 · (B,C)=7 · (A,C)=8 거부 · (B,D)=9 거부 · (C,E)=10 · (C,F)=12 거부 · (E,G)=14

Kruskal

```
Initialize: forest ← n개 정점, 간선 없음
Sort edges by cost (ascending) or use min-heap
for each edge (u, v) in order do
  r ← FIND(u);  s ← FIND(v)
  if r ≠ s then                ⟵ 다른 component → 사이클 없음
    add (u, v) to MST
    UNION(r, s)
  else                         ⟵ 같은 component → 사이클 → skip
    reject (u, v)
```

핵심 연산: FIND(어느 집합?) + UNION(두 집합 병합). 효율은 Union-Find 구현에 달림.



초기 5간선 채택 (G,H)·(C,D)·(A,B)·(E,F)·(B,C) (B,D)거부·(C,E)채택·(C,F)거부·(E,G)채택

MST 완성 — 7간선, 비용=49

Kruskal step-by-step — 빨강=채택 간선, 오름차순 검토·사이클 거부 (p.24-28)

✎ **핵심** MST 유일성: 간선 비용이 모두 distinct이면 MST는 유일. 교환 논증에서 동률이 없으면 매 cut의 최소 간선이 유일하게 결정되기 때문. 동률 존재 시 간선 구성이 다른 복수 MST 가능 (단 총비용은 동일).

Union-Find (Disjoint-Set) p.32-52

Q. 세 연산 MAKE/FIND/UNION은?

Q. naive linking 최악은?

Q. by-rank/size·path compression 효과?

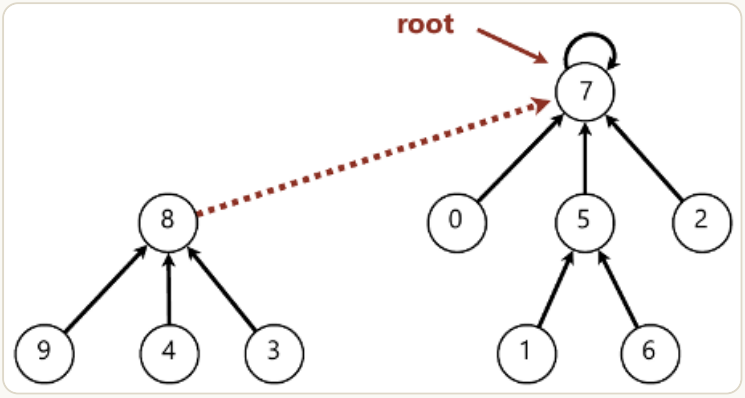
m=총 호출수
n=원소수

세 연산 — 서로소 집합 컬렉션에 대한 세 연산.

- MAKE-SET(x) — 원소 x만 든 새 집합 생성 (`parent[x] ← x`)
- FIND(x) — x가 속한 집합의 대표원소(루트) 반환 (parent 포인터 타고 올라감)
- UNION(x,y) — x·y의 집합을 합집합으로 교체 (루트 하나를 다른 루트 밑에 매달기)
- Kruskal 사이클 판정 = `FIND(u) == FIND(v)` ? 사이클 : 채택+UNION

Parent-link Representation — 각 집합을 *트리*로 표현. 루트가 대표원소(자기 자신을 가리킴). 배열 `parent[]` 로 구현.

- `parent[i] = j` ⟺ 원소 i의 부모가 j. 루트: `parent[i] = i`



UNION(3,5) — 왼쪽 트리 루트를 오른쪽 루트 7 밑에 매달기 (p.35)

Naïve Linking — 최악 Θ(n) — 루트를 그냥 다른 루트 밑에 매단다(크기·높이 무시). 최악: UNION(1,2), UNION(2,3), … → 일렬 사슬 → FIND가 Θ(n).

Naïve Linking

```
MAKE-SET(x):  parent[x] ← x
FIND(x):      while x ≠ parent[x]  do  x ← parent[x];  return x
UNION(x, y):  r ← FIND(x);  s ← FIND(y);  parent[r] ← s
```

크기·높이 고려 없음 → 트리가 한쪽으로 늘어져 최악 높이 n-1 발생.

Link-by-Size / Link-by-Rank — O(log n) — 두 최적화 모두 트리 높이를 O(log n)으로 묶어 UNION·FIND를 O(log n)으로 만든다.

- Link-by-size: 각 루트에 `size[]` (노드 수) 유지 — *작은 트리*를 *큰 트리* 루트 밑에
- UNION: size[r]>size[s] → parent[s]←r, size[r]+=size[s]; else 반대
- Link-by-rank: 각 노드에 `rank[]` (높이 추정치, 초기 0) 유지 — *낮은 rank*를 *높은 rank* 밑에
- 동률이면 한쪽 rank += 1 (동률일 때만 높이가 1 증가하므로 rank가 정확히 높이를 추정)

Link-by-size UNION

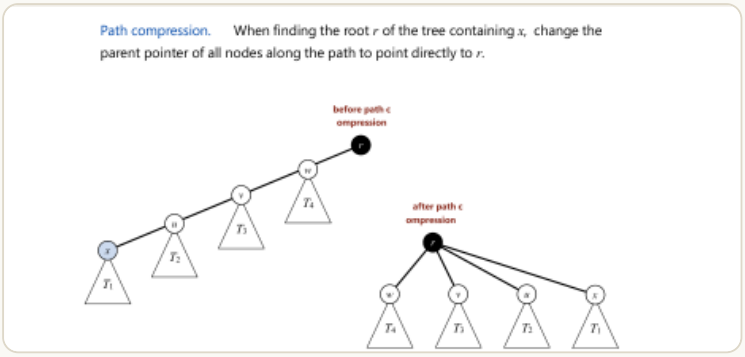
```
r ← FIND(x);  s ← FIND(y)
if r = s then return
if size[r] > size[s] then
  parent[s] ← r;  size[r] += size[s]
else
  parent[r] ← s;  size[s] += size[r]
```

Link-by-rank UNION

```
r ← FIND(x);  s ← FIND(y)
if r = s then return
if rank[r] > rank[s] then  parent[s]
← r
else if rank[r] < rank[s] then
parent[r] ← s
else
  parent[r] ← s;  rank[s] += 1
```

Path Compression — FIND(x)로 루트 r을 찾는 *김에* 경로상 모든 노드의 parent를 r로 직접 바꾼다. 이후 FIND가 O(1) 수준으로 빨라짐.

- 재귀 한 줄: `if x≠parent[x]: parent[x]←FIND(parent[x]); return parent[x]`
- 되돌아오는 길에 경로 노드 전부를 루트 직속으로 끌어올림 → 트리가 *납작*해짐



before/after path compression — 경로 노드 모두 루트 직속으로 (p.44)

복잡도 정리 — linking + path compression 결합 시 m번 연산이 O(m log n) (실질 역 Ackermann α(n) ≈ 상수).

- link-by-rank + path compression: O(m log n)
- link-by-size + path compression: O(m log n)
- Kruskal 전체: 간선 정렬 O(e log e) + 간선마다 FIND·UNION O(log e) × e번 = O(e log n)

응용 · 클러스터링 · MST vs 최단경로 p.53-65

Q. single-link clustering = ?

Q. MST ≠ 최단경로 트리인 이유?

Single-Link Clustering = Kruskal — k-clustering: 객체를 k개 그룹으로 나누되 *다른 클러스터끼리 거리 최대화*. Single-link(두 클러스터 간 거리 = 가장 가까운 쌍).

- 알고리즘: Kruskal을 실행하다 component 수가 k개가 되면 정지 → 그게 k-clustering
- MST의 비싼 k-1개 간선을 제거하면 곧 k-clustering — dendrogram으로 시각화

⚠ **함정** MST ≠ 최단경로 트리(Dijkstra). MST 목표 = *총 비용 합 최소* vs 최단경로 목표 = *source에서 각 정점까지 거리 최소*. MST 위의 두 정점 간 경로가 최단 경로라는 보장은 없다.

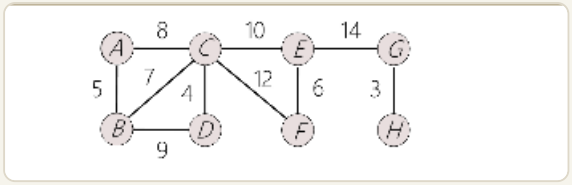
Sollin (Borůvka) 알고리즘 p.67-71

Q. Sollin 한 라운드 동작은?

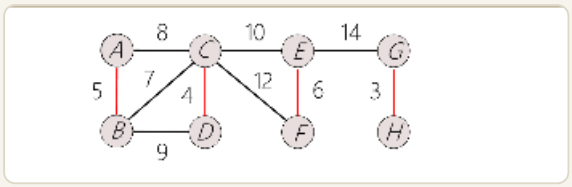
Q. 라운드 수와 강점(병렬)?

Sollin's Algorithm — n-정점 forest에서 각 component가 동시에 다른 component로의 최소 비용 간선 선택.

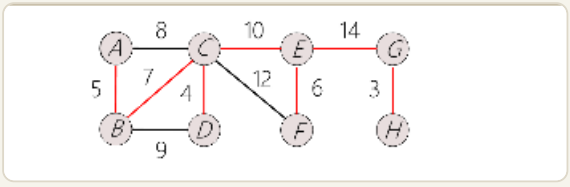
- 중복 선택·사이클 제거 → component 1개 남을 때까지 반복
- 매 라운드 component 수가 최소 절반으로 감소 → O(log n) 라운드
- Prim·Kruskal의 순차 진행과 달리 병렬화에 강점 — 각 component가 독립적



R1 선택 전: 각 정점이 최소 inter-cluster 간선 고름



R1 결과: 8→4 components ({A,B}·{C,D}·{E,F}·{G,H})



R2: {A,B}:7·{C,D}:7·{E,F}:10·{G,H}:14 → MST 완성

Sollin 라운드별 병합 — 동시에 최소 간선 선택, 중복 제거 (p.69-71)

Sollin 예제 (p.69-71) — 라운드 1: 각 정점이 자신의 최소 inter-cluster 간선 선택 → 쌍으로 겹쳐 4개 클러스터. 라운드 2: 4→1개.

- R1 선택: A=5·B=5 → {A,B}, C=4·D=4 → {C,D}, E=6·F=6 → {E,F}, G=3·H=3 → {G,H}
- R2 선택: {A,B}=7(B-C), {C,D}=7(B-C), {E,F}=10(C-E), {G,H}=14(E-G) → 1개

Prim vs Kruskal vs Sollin — 비교 p.66-74

Q. 시작·진행 단위 차이는?

Q. dense/sparse 어느 쪽 유리?

	Prim	Kruskal	Sollin
시작	1-정점 트리	n-정점 forest	n-정점 forest
진행 단위	정점 1개씩 추가	간선 1개씩 추가	component 전체 동시
자료구조	priority queue	Union-Find	Union-Find
복잡도	O(n²)/O(e log n)	**O(e log n)**	O(e log n)
dense	**유리 (O(V log V))**	보통	보통
sparse	보통	**유리 (단순 구조)**	보통
병렬화	불리	보통	**강점**
구현	어려움	**쉬움**	중간

dense→Prim(Fibonacci heap), sparse·구현 단순→Kruskal, 병렬→Sollin

✎ **핵심** 세 알고리즘 모두 cut property 기반 greedy이므로 같은 최소 비용의 MST를 만든다. 간선 비용 동률이 있으면 간선 구성이 다른 복수 MST 가능(총비용은 동일). Euclidean MST: brute-force O(N²) → *Delaunay 삼각분할* 이용 시 O(N log N).

★ **필수암기** MST=가중연결무방향·n-1간선·총비용최소 · Prim=정점확장·D[.]+P[.]+Q·O(n²)/O(e log n) · cut property=최소비용cross-cut간선⊆MST·귀류법 · 음수간선=Prim도OK(MST는총비용) · Kruskal=간선오름차순·사이클skip·O(e log n) · Union-Find: naive=Θ(n)·by-rank/size=O(log n)·path-compression+linking=O(m log n) · Sollin=병렬·O(log n)라운드 · single-link clustering=Kruskal k정지

요약 MST = 가중 연결 무방향에서 총비용 최소 스패닝 트리, 모두 **cut property** 기반 greedy · Prim(정점 확장·D[.]+P[.]·O(n²)/O(e log n)) · Kruskal(간선 오름차순·사이클 skip·Union-Find O(e log n)) · Union-Find(by-rank/size+path compression≈O(m·α)) · Sollin(병렬·O(log n) 라운드).

Ch 17 String — 문자열 인덱싱

Trie → Compressed Trie → Suffix Tree → Exact Matching → Generalized → Suffix Array

String · Trie · Compressed Trie p.1-7

Q. string 정의와 char 인코딩?
Q. trie의 검색 복잡도는?
Q. compressed trie 압축 방식은?

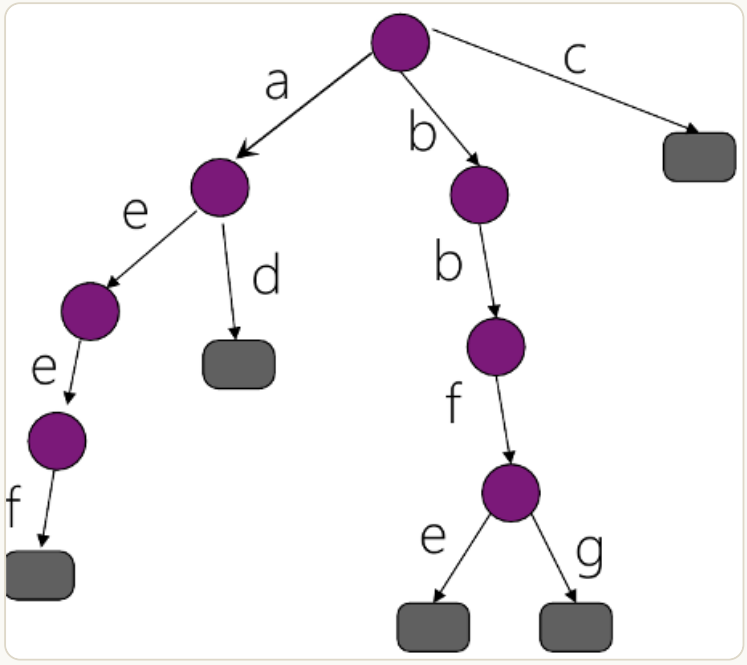
String (문자열) 정의 — 유한 알파벳 위 문자들의 순서 있는 나열 (인덱스 1…m). 순서가 있어야 substring·prefix·suffix가 정합적으로 정의됨.

- 도메인 독립: 영어·DNA(*GATC*)·코드 — 알파벳 크기만 다를 뿐 *같은 알고리즘* 적용
- char: C=8b ASCII, Java=16b Unicode, Python=내부 Unicode(인코딩 은닉)
- Unicode — 코드 포인트(code point) vs 바이트 표현(UTF-8/16/32) 구분 필수

Trie — 문자열 *집합* 위의 트리 — 예: {aeef, ad, bbfe, bbfg, c}. 접두사 공유로 O(L) 검색.

O(L) 검색
집합 크기 무관

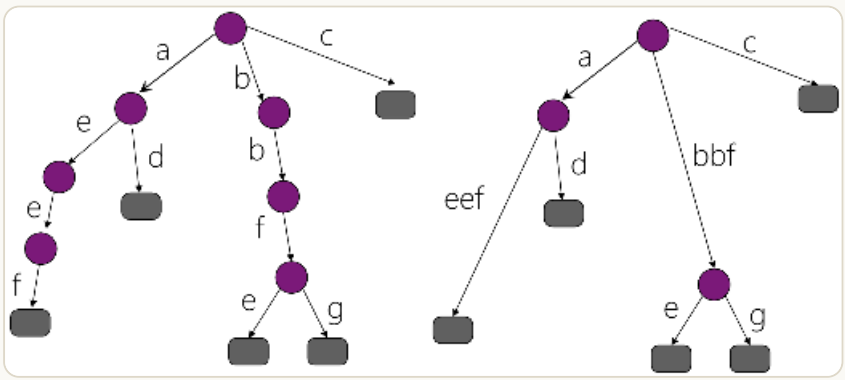
- 전제: 집합 내 어떤 문자열도 다른 것의 접두사(prefix)가 아님 — 위반 시 leaf:1 대응 붕괴
- 간선 라벨 = 한 글자, *같은 노드의 두 간선은 동일 라벨 불가*



Trie 예제: {aeef, ad, bbfe, bbfg, c} (p.6)

Compressed Trie — 자식이 *하나뿐인 노드* 사슬을 합치고 라벨을 부분문자열로 연결. 노드 수·깊이 감소, 점근 복잡도 동일.

- 간선 저장 = `(start, end)` 포인터 — 부분문자열 중복 없이 표현
- 분기 판단: "어느 자식?" → "어느 자식, 라벨 안 어디까지?"로 확장



Compressed Trie: 사슬 합치기 전후 비교 (p.7)

Suffix Tree — 정의 p.8-11

Q. suffix tree란?
Q. sentinel $의 역할은?
Q. leaf 수·내부 노드 자식 수?

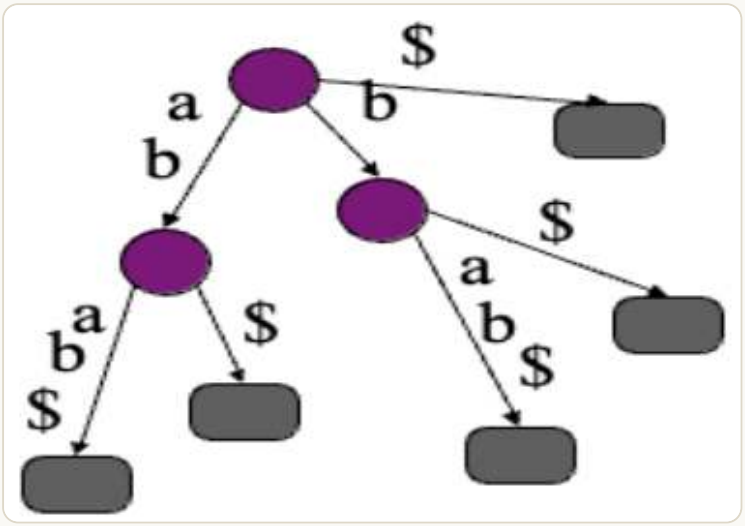
Suffix Tree — 텍스트 T의 **모든 접미사** 위 Compressed Trie. 한 번 구축 → 이후 패턴 질의가 텍스트 크기와 무관.

m=텍스트 길이
leaf=접미사 위치

- 정확히 **m개 leaf**, 각 내부 노드(root 제외) 자식 ≥ 2
- 각 간선 라벨 = T의 *비어있지 않은 부분 문자열*
- **첫 글자 규칙**: 같은 노드에서 두 간선은 *같은 첫 글자 불가* → 분기가 *결정적*
- leaf i의 root→leaf 경로를 이으면 정확히 T[i,⋯,m]을 출력

센티널 $ — T 끝에 알파벳에 없는 특수 문자 `$` 추가 → 어떤 접미사도 다른 접미사의 **접두사가 되지 않음** 보장 → 모든 접미사가 leaf에서 끝남.

- 압축 후에도 **O(m) 노드**만 유지 → 실용적 인덱스 가능



Suffix Tree(T=abab$) 완성 구조 — leaf 번호=시작 위치 (p.11)

Naive Construction — 구축 알고리즘 p.12-16

Q. 순차 삽입(긴 접미사부터) 방식?
Q. 시간·공간 복잡도는?
Q. 선형 구축 알고리즘은?

순차 삽입 (Naive) — T=abab$. 접미사를 *긴 것부터* 하나씩 삽입 — 기존 트리에서 공통 접두사 경로를 따라가다 분기.

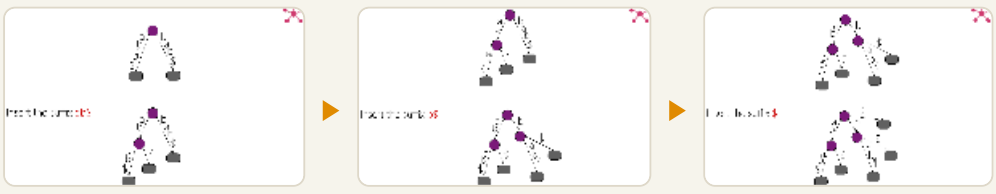
삽입당 O(m)
총 O(m²)

- 삽입 1개: O(m) 경로 걷기 × m회 = **O(m²)** 총 비용
- 공간은 공유 덕분에 **O(m) 노드** 유지

Naive Suffix Tree 구축

```
Insert T[1..m]$ into empty tree   ← 전체 문자열(가장 긴 접미사)
for i := 2 to m+1 do
  walk from root consuming T[i..m]$
  if mismatch mid-edge then
    split edge, create new internal node
  create new leaf for suffix T[i..m]$
```

O(m) 삽입 × m회 = O(m²). Weiner·McCreight·Ukkonen은 suffix link로 O(m) 달성.



① ab$ 삽입 — ab 공유 ② b$ 삽입 ③ $ 삽입 → 완성

T=abab$ Naive 구축 — 긴 접미사(abab$·bab$)부터 차례로 삽입 (p.15-17)

✎ **핵심** 슬라이드 leaf 번호: 1=abab$, 2=bab$, 3=ab$, 4=b$, 5=$. 구축 후 **leaf i ↔ 시작위치 i** 대응 확인이 정의 검증의 핵심.

Exact String Matching p.17-21

Q. 매칭 방법(트리 따라가기)?
Q. 시간 복잡도 O(m+n+k)의 구성?
Q. 출현 위치 k는 어떻게 수집?

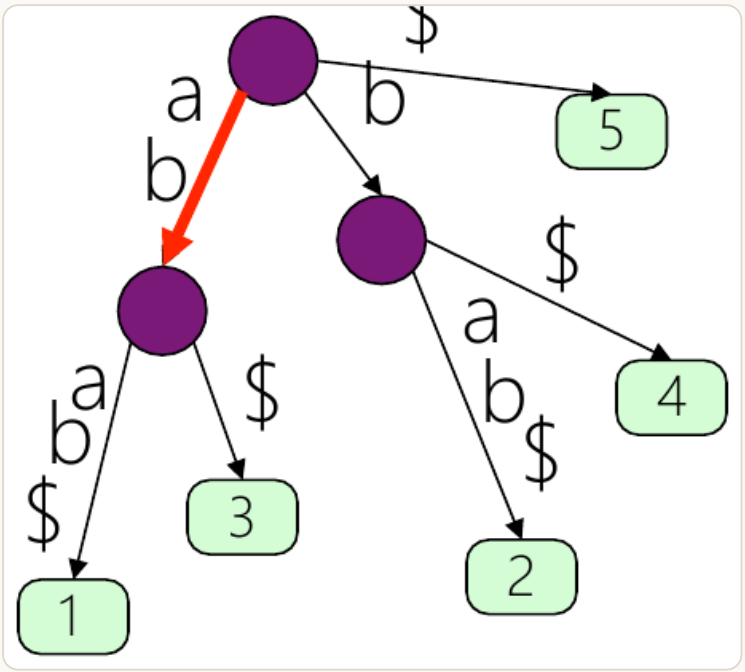
Suffix Tree 기반 매칭 — T에 대한 suffix tree를 미리 구축 → 패턴 s(|s|=n)를 root에서 **트리 경로를 따라 내려가며** 소비.

패턴 소비 후 서브트리 수집

- s 소진 성공 → 도달 노드의 **서브트리 leaf 전부** = 모든 출현 위치 k개
- 다음 글자에 해당하는 간선 없음 → *출현 없음*

시간 복잡도 O(m+n+k) — 세 단계 분해:

- **구축**: O(m) — Weiner·McCreight·Ukkonen 선형 알고리즘
- **매칭**: O(n), n=|패턴| — 경로 내려가기
- **열거**: O(k), k=출현 횟수 — leaf 수집
- 분할상환 우위: **m은 일회성 전처리**, 반복 질의는 n+k만 지불



aba ⊂ abab$? — root→a→b 경로 후 서브트리 leaf={1,3} (p.21-22)

⚠ **함정** Naive 구축 O(m²) vs 선형 구축 O(m): *결과 트리는 동일*. naive는 개념 이해용, 실무는 O(m) 알고리즘 사용.

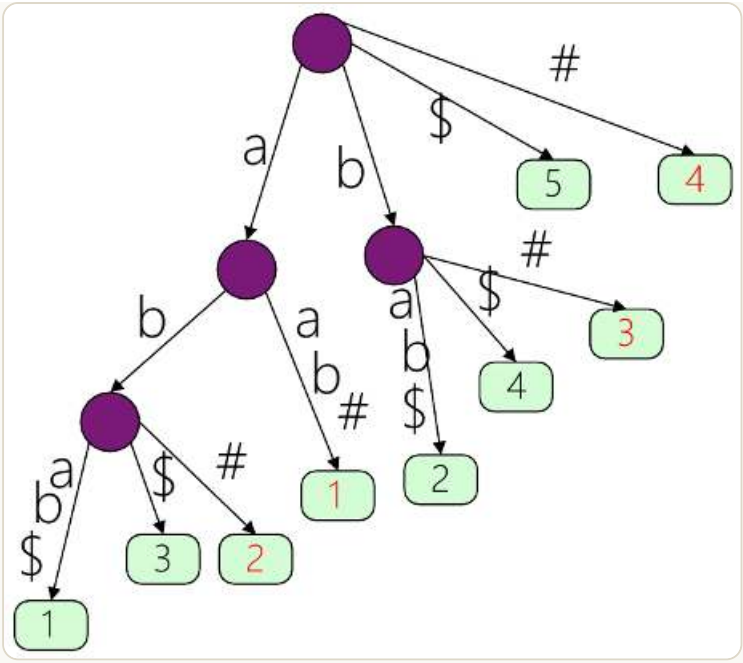
Generalized Suffix Tree — 다중 문자열 p.22-24

Q. 여러 문자열을 어떻게 한 트리에?
Q. 서로 다른 sentinel이 필요한 이유?

Generalized Suffix Tree — 문자열 컬렉션 $T=\{t_1,\cdots,t_k\}$의 *모든 접미사 합집합* 위 단일 compressed trie.

컬렉션 전체 단일 트리

- 각 t_i마다 **서로 다른 센티널** 부착 (예: $, #) → leaf 출처 구분
- leaf = (t_i, position) 쌍으로 *정체성 모호 없이* 유지
- 공간: 모든 입력 크기 합에 *선형* — 상수 인자 큰 것이 단점



Generalized suffix tree: t_1=abab, t_2=aab — $ / # 센티널로 구분 (p.27)

Suffix Array — 경량 대안 p.25-33

Q. suffix array 정의는?

Q. 검색(이진탐색) 복잡도는?

Q. suffix tree 대비 장단점?

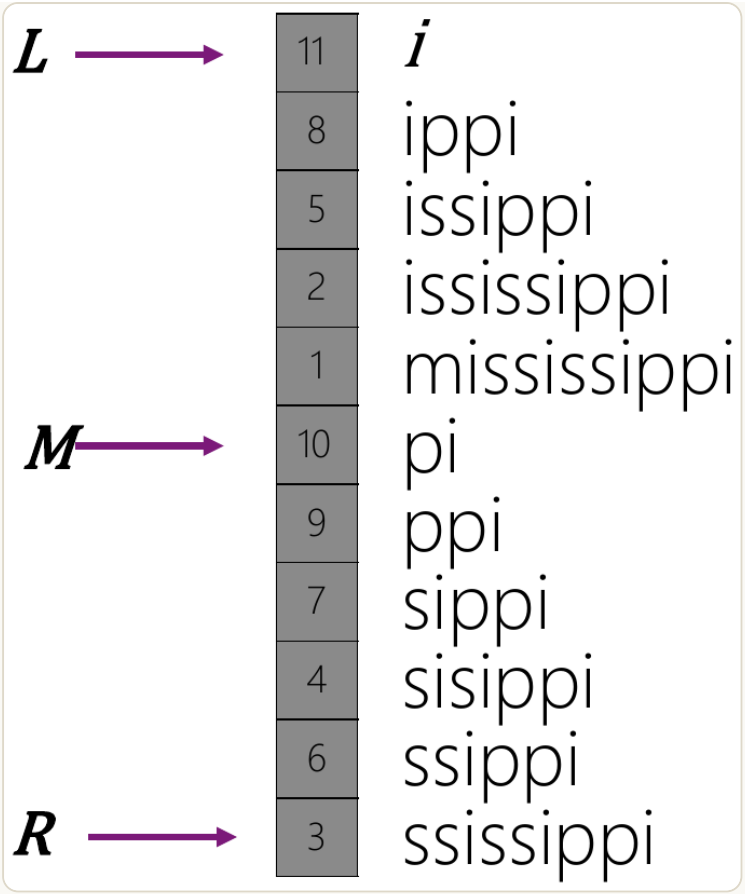
Suffix Array 정의 — 모든 접미사의 시작 위치를 **사전식(lexicographic) 순서**로 정렬해 저장한 정수 배열.

정수 배열 = 사전순 위치

- T=abab → 정렬 순서: ab < abab < b < bab → 배열 **[3, 1, 4, 2]**
- 노드 대신 *위치당 정수 하나*만 저장 → suffix tree 대비 대폭 경량
- 배열 원소의 정렬 순서가 접두사 공유 정보를 *암시적으로* 인코딩

검색 — 이진 탐색 — 패턴 s의 모든 출현은 정렬된 배열에서 **연속 구간**으로 모임 → 이진 탐색으로 구간 확정.

- s 이상인 첫 위치 L, s 초과인 첫 위치 R 탐색 → [L,R) 구간
- 시간: **O(n log m)** — 문자 비교 O(n) × 이진탐색 log m회
- Suffix tree O(n+k) 대비 느리지만 *캐시 친화적·직렬화 용이*



mississippi suffix array: s=issa → L=index of issippi, R=sippi (p.26)

Suffix Array 구축 방법

- **방법 1 (트리 경유)**: suffix tree DFS로 leaf 수집 → `O(m)` (트리 구조 먼저 필요)
- **방법 2 (직접 정렬)**: 정렬 기반 → `O(m·r·log m)`, r=최장 반복 길이
- 현대 라이브러리(SA-IS, divsufsort): `O(m)` 직접 구성

비교 · 정리 p.31-33

Q. Trie/Compressed/SuffixTree/Array 비교?

Q. 시간 우선 vs 공간 우선 선택은?

구조	전처리	질의	공간	특징
Trie	O(총 길이)	O(L)	O(총 글자)	집합·접두사 공유
Compressed Trie	O(총 길이)	O(L)	O(문자열 수)	사슬 합치기
Suffix Tree	O(m) *선형	O(n+k)	O(m) *상수 큼	단일 텍스트 인덱스
Suffix Array	O(m)	O(n log m)	O(m) *경량	이진탐색, 캐시↑

*선형 구축은 Weiner/McCreight/Ukkonen. n=패턴 길이, k=출현 횟수, m=텍스트 길이

✎ **핵심** 선택 기준: 시간 우선 → Suffix Tree O(n+k) / 공간 우선 → Suffix Array O(n log m) / 다중 텍스트 → Generalized suffix tree (서로 다른 센티널 필수)

★ **필수암기** Trie=집합 트리·접두사없음 전제·O(L) / CompressedTrie=사슬압축·(start,end) 포인터 / SuffixTree=T 전체 접미사 compressed trie·sentinel $·m leaf·O(m) 구축·O(n+k) 매칭 / NaiveO(m²) / Exact match=트리 따라가다 서브트리 leaf / SuffixArray=[정수배열·사전순]·이진탐색·O(n log m)

요약 문자열 인덱스 = Trie(집합·접두사 공유·O(L) 검색) → Compressed Trie(사슬 압축·(start,end) 포인터) → **Suffix Tree**(T의 모든 접미사 compressed trie·sentinel $·m leaf·구축 O(m)·매칭 O(n+k)) → **Suffix Array**([정수배열·사전순]·이진탐색 O(n log m)·경량) · 다중=Generalized(서로 다른 sentinel).